Minimal MNIST Rectified Flow Notebook with Reflow, and One-Step Distillation

This notebook implements a **minimal unconditional MNIST** version of the Rectified Flow pipeline. It starts with a standard CFM teacher, performs **reflow** to obtain a **2-Rectified Flow**, optionally repeats reflow for **(k=3)**, and then adds the paper's **one-step distillation** stage after (k=2) and optionally after (k=3).

Along the way, the notebook tracks three flow-side metrics:

1. the **straightness measure**,
2. the extra **endpoint-consistency** metric,
3. the **quadratic transport cost** (\mathbb{E}|Z_1-Z_0|^2), which is a special case of the paper's non-increasing convex transport cost viewpoint.

For distillation, the notebook then tracks the separate **one-step endpoint MSE** against the source flow's endpoints.

Block 1 — Imports and Global Configuration

This cell imports the required libraries, sets the random seed, chooses the device, and defines the main hyperparameters used throughout the notebook.

For A100 run, the default schedule in this version is **50 CFM epochs**, **20 reflow epochs**, and **12 distillation epochs**. The notebook also smooths the metric curves and evaluates them on larger Monte Carlo batches so the plots are less noisy.

In [ ]:
```
# Block 1: Imports and global config

import copy
import json
import math
import random
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.optim import AdamW
```

```
from torch.utils.data import DataLoader
from torchvision.datasets import MNIST
import torchvision.transforms as transforms
from torchvision.utils import make_grid, save_image
from tqdm.auto import tqdm

# ----------------------------
# User config
# ----------------------------
SEED = 1234
DATA_ROOT = "./datasets"
OUT_DIR = Path("./mnist_rectified_flow_notebook_runs")
OUT_DIR.mkdir(parents=True, exist_ok=True)

BATCH_SIZE = 128
HIDDEN_DIM = 128
N_LAYERS = 6
LR = 1e-4

CFM_EPOCHS = 50
REFLOW_EPOCHS = 20
DISTILL_EPOCHS = 12   # final one-step distillation after a rectified

PAIR_STEPS = 32          # teacher solver steps used to generate reflo
PAIR_SOLVER = "midpoint" # better for teacher pair generation
SAMPLE_SOLVER = "euler"  # used for low-NFE sample sweeps
SAMPLE_STEPS = [1, 2, 4, 8, 16]

VIS_N = 64
EVAL_BATCH_SIZE = 256

RUN_OPTIONAL_K3 = True   # pushed to the end of the notebook

# Checkpoint-tracking config for metric curves
TRACK_METRICS = True
TRACK_METRIC_EVERY = 2
TRACK_STRAIGHTNESS_BATCHES = 6
TRACK_ENDPOINT_BATCHES = 4
TRACK_ENDPOINT_STEPS = 16
FINAL_METRIC_BATCHES = 10
HISTORY_SMOOTHING_WINDOW = 3
SAVE_EPOCH_CHECKPOINTS = True

NUM_WORKERS = 0
SHOW_SAVED_SAMPLE_GRIDS = True
INLINE_SAMPLE_STEPS = [1, 2, 4, 16]

SHOW_TQDM = True
TQDM_UPDATE_EVERY = 20

def set_seed(seed: int) -> None:
    random.seed(seed)
```

```
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
Device: cuda
```

Block 2 — MNIST Data Loader

This cell downloads MNIST, applies the minimal preprocessing needed for unconditional generation, and builds the training dataloader. The rest of the notebook assumes images are normalized into the same tensor format returned here.

In [ ]:
```
# Block 2: Data loader

def build_mnist_loader(data_root: str, batch_size: int, device: torch.
    transform = transforms.Compose([transforms.ToTensor()])
    dataset = MNIST(data_root, train=True, download=True, transform=tr
    kwargs = {"num_workers": NUM_WORKERS, "pin_memory": device.type ==
    return DataLoader(dataset, batch_size=batch_size, shuffle=True, dr

train_loader = build_mnist_loader(DATA_ROOT, BATCH_SIZE, device)
print("Number of batches per epoch:", len(train_loader))
print("NUM_WORKERS =", NUM_WORKERS)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 43.8MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.05MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 10.0MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 12.9MB/s]
Number of batches per epoch: 468
NUM_WORKERS = 0
```

Block 3 — Velocity Model for MNIST

This cell defines the convolutional velocity network used to parameterize the flow on images. It is kept close to the original MNIST notebook so that the main new ingredient in this notebook is the rectification procedure, not a major model redesign.

In [ ]:
```
# Block 3: Model (kept close to the MNIST notebook)

class ConvBlock(nn.Conv2d):
    def __init__(
        self,
        in_channels: int,
```

```
        out_channels: int,
        kernel_size: int,
        activation_fn: bool = False,
        stride: int = 1,
        padding: str | int = "same",
        dilation: int = 1,
        groups: int = 1,
        bias: bool = True,
        gn: bool = False,
        gn_groups: int = 8,
    ):
        if padding == "same":
            padding = (kernel_size // 2) * dilation
        super().__init__(
            in_channels,
            out_channels,
            kernel_size,
            stride=stride,
            padding=padding,
            dilation=dilation,
            groups=groups,
            bias=bias,
        )
        self.activation_fn = nn.SiLU() if activation_fn else None
        self.group_norm = nn.GroupNorm(gn_groups, out_channels) if gn

    def forward(self, x: torch.Tensor, time_embedding: torch.Tensor |
        if residual:
            if time_embedding is None:
                raise ValueError("time_embedding is required when resi
            x = x + time_embedding
            y = x
            x = super().forward(x)
            y = y + x
        else:
            y = super().forward(x)

        if self.group_norm is not None:
            y = self.group_norm(y)
        if self.activation_fn is not None:
            y = self.activation_fn(y)
        return y


class ImageFlow(nn.Module):
    def __init__(self, image_resolution=(28, 28, 1), hidden_dims=(128,
        super().__init__()
        _, _, img_c = image_resolution
        hidden_dims = list(hidden_dims)
        if len(hidden_dims) < 2:
            raise ValueError("Need at least two hidden dimensions.")
```

```
        self.in_project = ConvBlock(img_c, hidden_dims[0], kernel_size
        self.time_project = nn.Sequential(
            ConvBlock(1, hidden_dims[0], kernel_size=1, activation_fn=
            ConvBlock(hidden_dims[0], hidden_dims[1], kernel_size=1),
        )

        self.convs = nn.ModuleList([ConvBlock(hidden_dims[0], hidden_d
        for idx in range(1, len(hidden_dims)):
            dilation = 3 ** ((idx - 1) // 2)
            self.convs.append(
                ConvBlock(
                    hidden_dims[idx - 1],
                    hidden_dims[idx],
                    kernel_size=3,
                    dilation=dilation,
                    activation_fn=True,
                    gn=True,
                    gn_groups=8,
                )
            )

        self.out_project = ConvBlock(hidden_dims[-1], img_c, kernel_si

    @staticmethod
    def interpolate(x0: torch.Tensor, x1: torch.Tensor, t: torch.Tenso
        return (1.0 - t) * x0 + t * x1

    @staticmethod
    def target_velocity(x0: torch.Tensor, x1: torch.Tensor) -> torch.T
        return x1 - x0

    def forward(self, x: torch.Tensor, t: torch.Tensor) -> torch.Tenso
        if t.ndim == 1:
            t = t[:, None, None, None]
        elif t.ndim == 2:
            t = t[:, :, None, None]

        time_embedding = self.time_project(t)
        y = self.in_project(x)
        for block in self.convs:
            y = block(y, time_embedding, residual=True)
        return self.out_project(y)


def count_parameters(model: nn.Module) -> int:
    return sum(p.numel() for p in model.parameters() if p.requires_gra

hidden_dims = [HIDDEN_DIM for _ in range(N_LAYERS)]
teacher = ImageFlow(image_resolution=(28, 28, 1), hidden_dims=hidden_d
print("Trainable parameters:", f"{count_parameters(teacher):,}")
```

```
Trainable parameters: 911,105
```

Block 4 — ODE Integration and Sampling Helpers

This cell defines the numerical integrator used to evolve samples from noise to data. These helpers are reused both for generating samples at different NFEs and for constructing teacher-generated endpoint pairs during reflow.

In [ ]:
```
# Block 4: ODE integration and sampling helpers

@torch.no_grad()
def integrate_flow(
    model: nn.Module,
    x0: torch.Tensor,
    num_steps: int,
    solver: str = "euler",
    return_path: bool = False,
):
    if num_steps < 1:
        raise ValueError("num_steps must be >= 1")

    model.eval()
    x = x0.clone()
    if return_path:
        path = [x.clone()]

    if num_steps == 1:
        t0 = torch.zeros(x.shape[0], 1, 1, 1, device=x.device, dtype=x
        x = x + model(x, t0)
        if return_path:
            path.append(x.clone())
            return torch.stack(path, dim=0)
        return x

    dt = 1.0 / num_steps
    t_scalar = 0.0
    for _ in range(num_steps):
        if solver == "euler":
            t = torch.full((x.shape[0], 1, 1, 1), t_scalar, device=x.d
            v = model(x, t)
            x = x + dt * v
        elif solver == "midpoint":
            t = torch.full((x.shape[0], 1, 1, 1), t_scalar, device=x.d
            v1 = model(x, t)
            x_mid = x + 0.5 * dt * v1
            t_mid = torch.full((x.shape[0], 1, 1, 1), t_scalar + 0.5 *
            v2 = model(x_mid, t_mid)
            x = x + dt * v2
        else:
            raise ValueError("solver must be 'euler' or 'midpoint'")

        t_scalar += dt
        if return_path:
```

```
            path.append(x.clone())

    if return_path:
        return torch.stack(path, dim=0)
    return x


@torch.no_grad()
def sample_images(
    model: nn.Module,
    n_samples: int,
    batch_size: int,
    num_steps: int,
    device: torch.device,
    solver: str = "euler",
):
    outs = []
    remaining = n_samples
    while remaining > 0:
        cur = min(batch_size, remaining)
        x0 = torch.randn(cur, 1, 28, 28, device=device)
        x1 = integrate_flow(model, x0, num_steps=num_steps, solver=sol
        outs.append(x1.cpu())
        remaining -= cur
    return torch.cat(outs, dim=0)


@torch.no_grad()
def save_sample_grid(
    samples: torch.Tensor,
    out_path: Path,
    nrow: int = 8,
    show_inline: bool = False,
    title: str | None = None,
):
    grid = make_grid(samples.clamp(0, 1), nrow=nrow)
    save_image(grid, out_path)
    print("Saved:", out_path)
    if show_inline:
        grid_np = grid.detach().cpu().numpy()
        grid_np = np.transpose(grid_np, (1, 2, 0))
        plt.figure(figsize=(6, 6))
        if grid_np.shape[-1] == 1:
            plt.imshow(grid_np[..., 0], cmap="gray")
        else:
            plt.imshow(grid_np)
        if title is not None:
            plt.title(title)
        plt.axis("off")
        plt.tight_layout()
        plt.show()
```

Block 5 — Training Batch Construction

This cell defines how training triples are built for the two stages. For stage 1, it uses the standard CFM interpolation between Gaussian noise and real MNIST images; for reflow, it builds new pairs from trajectories generated by the previous model.

In [ ]:
```
# Block 5: Training batch samplers

def sample_cfm_training_batch(x1: torch.Tensor):
    x0 = torch.randn_like(x1)
    t = torch.rand(x1.shape[0], 1, 1, 1, device=x1.device, dtype=x1.dt
    x_t = ImageFlow.interpolate(x0, x1, t)
    target = ImageFlow.target_velocity(x0, x1)
    return x_t, t, target


@torch.no_grad()
def sample_reflow_training_batch(
    teacher: nn.Module,
    batch_size: int,
    device: torch.device,
    pair_steps: int,
    pair_solver: str,
):
    x0 = torch.randn(batch_size, 1, 28, 28, device=device)
    x1 = integrate_flow(teacher, x0, num_steps=pair_steps, solver=pair
    t = torch.rand(batch_size, 1, 1, 1, device=device)
    x_t = ImageFlow.interpolate(x0, x1, t)
    target = ImageFlow.target_velocity(x0, x1)
    return x_t, t, target
```

Block 5A — Metric Helpers

This cell defines the three evaluation metrics used later in the notebook.

- The first is the **straightness measure**.
- The second is the **endpoint-consistency** measure
- The third is the **quadratic transport cost** (\mathbb{E}|Z_1-Z_0|^2), included as a special case of the paper's convex transport-cost perspective.

To reduce Monte Carlo noise, the endpoint-consistency metric is averaged across multiple independent trajectory batches, and the checkpoint curves later use a small moving-average smoother for visualization.

In [ ]:
```
# Block 5A: Metric helpers (defined early so training can track checkp

@torch.no_grad()
```

```python
def estimate_paper_straightness(
    model: nn.Module,
    device: torch.device,
    batch_size: int = 128,
    pair_steps: int = 32,
    pair_solver: str = "midpoint",
    batches: int = 8,
):
    model.eval()
    vals = []
    for _ in range(batches):
        x0 = torch.randn(batch_size, 1, 28, 28, device=device)
        x1 = integrate_flow(model, x0, num_steps=pair_steps, solver=pa
        t = torch.rand(batch_size, 1, 1, 1, device=device)
        x_t = ImageFlow.interpolate(x0, x1, t)
        chord = x1 - x0
        pred = model(x_t, t)
        vals.append(F.mse_loss(pred, chord).item())
    return float(np.mean(vals))


@torch.no_grad()
def estimate_endpoint_consistency(
    model: nn.Module,
    device: torch.device,
    batch_size: int = 64,
    num_steps: int = 16,
    solver: str = "euler",
    batches: int = 4,
):
    model.eval()
    batch_vals = []
    for _ in range(batches):
        x0 = torch.randn(batch_size, 1, 28, 28, device=device)
        path = integrate_flow(model, x0, num_steps=num_steps, solver=s
        z1 = path[-1]
        errs = []

        for i in range(num_steps):
            t_scalar = i / num_steps
            zt = path[i]
            t = torch.full((batch_size, 1, 1, 1), t_scalar, device=dev
            vt = model(zt, t)
            z1_hat = zt + (1.0 - t_scalar) * vt
            errs.append(F.mse_loss(z1_hat, z1).item())

        batch_vals.append(float(np.mean(errs)))
    return float(np.mean(batch_vals))


@torch.no_grad()
def estimate_quadratic_transport_cost(
```

```
    model: nn.Module,
    device: torch.device,
    batch_size: int = 128,
    pair_steps: int = 32,
    pair_solver: str = "midpoint",
    batches: int = 8,
):
    model.eval()
    vals = []
    for _ in range(batches):
        x0 = torch.randn(batch_size, 1, 28, 28, device=device)
        x1 = integrate_flow(model, x0, num_steps=pair_steps, solver=pa
        disp = (x1 - x0).flatten(1)
        vals.append((disp.pow(2).sum(dim=1).mean()).item())
    return float(np.mean(vals))


@torch.no_grad()
def evaluate_metric_bundle(
    model: nn.Module,
    device: torch.device,
    straightness_batch_size: int = 128,
    straightness_pair_steps: int = 32,
    straightness_pair_solver: str = "midpoint",
    straightness_batches: int = 4,
    endpoint_batch_size: int = 128,
    endpoint_steps: int = 16,
    endpoint_solver: str = "euler",
    endpoint_batches: int = 2,
    transport_batch_size: int = 128,
    transport_pair_steps: int = 32,
    transport_pair_solver: str = "midpoint",
    transport_batches: int = 4,
):
    return {
        "paper_straightness": estimate_paper_straightness(
            model,
            device=device,
            batch_size=straightness_batch_size,
            pair_steps=straightness_pair_steps,
            pair_solver=straightness_pair_solver,
            batches=straightness_batches,
        ),
        "endpoint_consistency": estimate_endpoint_consistency(
            model,
            device=device,
            batch_size=endpoint_batch_size,
            num_steps=endpoint_steps,
            solver=endpoint_solver,
            batches=endpoint_batches,
        ),
        "quadratic_transport_cost": estimate_quadratic_transport_cost(
```

```
            model,
            device=device,
            batch_size=transport_batch_size,
            pair_steps=transport_pair_steps,
            pair_solver=transport_pair_solver,
            batches=transport_batches,
        ),
    }
```

Block 6 — Training Functions

This cell contains the training loops for the stage-1 CFM model and the later reflow stages. It also supports checkpoint-wise metric tracking so we can plot straightness and endpoint-consistency over training.

In [ ]:
```
# Block 6: Training functions

def maybe_tqdm(iterable, enabled: bool = True, **kwargs):
    return tqdm(iterable, **kwargs) if enabled else iterable


def train_cfm_stage(
    model: nn.Module,
    loader: DataLoader,
    device: torch.device,
    epochs: int,
    lr: float,
    stage_name: str = "stage1_cfm",
    track_metrics: bool = True,
    track_metric_every: int = 1,
    track_straightness_batches: int = 6,
    track_endpoint_batches: int = 4,
    track_endpoint_steps: int = 16,
    checkpoint_dir: Path | None = None,
):
    optimizer = AdamW(model.parameters(), lr=lr, betas=(0.9, 0.99))
    losses = []
    history = []
    model.train()

    if checkpoint_dir is not None:
        checkpoint_dir.mkdir(parents=True, exist_ok=True)

    for epoch in range(epochs):
        running = 0.0
        count = 0
        batch_iter = maybe_tqdm(
            loader,
            enabled=SHOW_TQDM,
            desc=f"{stage_name} | epoch {epoch + 1}/{epochs}",
            leave=False,
```

```
            dynamic_ncols=True,
        )
        for batch_idx, (x1, _) in enumerate(batch_iter, start=1):
            x1 = x1.to(device)
            x_t, t, target = sample_cfm_training_batch(x1)

            optimizer.zero_grad(set_to_none=True)
            pred = model(x_t, t)
            loss = F.mse_loss(pred, target)
            loss.backward()
            optimizer.step()

            losses.append(loss.item())
            running += loss.item()
            count += 1

            if SHOW_TQDM and hasattr(batch_iter, "set_postfix") and (
                batch_idx % TQDM_UPDATE_EVERY == 0 or batch_idx == len
            ):
                batch_iter.set_postfix(batch_loss=f"{loss.item():.4f}"

        avg = running / max(1, count)
        entry = {"epoch": epoch + 1, "avg_loss": float(avg)}

        should_track = track_metrics and (((epoch + 1) % track_metric_
        if should_track:
            metrics = evaluate_metric_bundle(
                model,
                device=device,
                straightness_batch_size=EVAL_BATCH_SIZE,
                straightness_pair_steps=PAIR_STEPS,
                straightness_pair_solver=PAIR_SOLVER,
                straightness_batches=track_straightness_batches,
                endpoint_batch_size=EVAL_BATCH_SIZE,
                endpoint_steps=track_endpoint_steps,
                endpoint_solver=SAMPLE_SOLVER,
                endpoint_batches=track_endpoint_batches,
                transport_batch_size=EVAL_BATCH_SIZE,
                transport_pair_steps=PAIR_STEPS,
                transport_pair_solver=PAIR_SOLVER,
                transport_batches=max(1, track_straightness_batches),
            )
            entry.update(metrics)
            print(
                f"[{stage_name}] epoch {epoch + 1}/{epochs} - loss: {a
                f"straightness: {metrics['paper_straightness']:.6f} |
                f"endpoint_consistency: {metrics['endpoint_consistency
                f"quadratic_transport_cost: {metrics['quadratic_transp
            )
        else:
            print(f"[{stage_name}] epoch {epoch + 1}/{epochs} - loss:
```

```
        history.append(entry)

        if checkpoint_dir is not None:
            torch.save(
                {
                    "model": model.state_dict(),
                    "losses": losses,
                    "history": history,
                    "epoch": epoch + 1,
                    "stage_name": stage_name,
                },
                checkpoint_dir / f"{stage_name}_epoch_{epoch + 1:02d}.
            )

    return model, losses, history


def train_reflow_stage(
    teacher: nn.Module,
    loader: DataLoader,
    device: torch.device,
    epochs: int,
    lr: float,
    stage_name: str,
    pair_steps: int,
    pair_solver: str,
    track_metrics: bool = True,
    track_metric_every: int = 1,
    track_straightness_batches: int = 6,
    track_endpoint_batches: int = 4,
    track_endpoint_steps: int = 16,
    checkpoint_dir: Path | None = None,
):
    student = copy.deepcopy(teacher).to(device)
    teacher.eval()
    student.train()

    optimizer = AdamW(student.parameters(), lr=lr, betas=(0.9, 0.99))
    losses = []
    history = []

    if checkpoint_dir is not None:
        checkpoint_dir.mkdir(parents=True, exist_ok=True)

    batch_size = loader.batch_size or 128
    for epoch in range(epochs):
        running = 0.0
        count = 0
        batch_iter = maybe_tqdm(
            loader,
            enabled=SHOW_TQDM,
            desc=f"{stage_name} | epoch {epoch + 1}/{epochs}",
```

```
            leave=False,
            dynamic_ncols=True,
        )
        for batch_idx, (_x, _) in enumerate(batch_iter, start=1):
            x_t, t, target = sample_reflow_training_batch(
                teacher=teacher,
                batch_size=batch_size,
                device=device,
                pair_steps=pair_steps,
                pair_solver=pair_solver,
            )

            optimizer.zero_grad(set_to_none=True)
            pred = student(x_t, t)
            loss = F.mse_loss(pred, target)
            loss.backward()
            optimizer.step()

            losses.append(loss.item())
            running += loss.item()
            count += 1

            if SHOW_TQDM and hasattr(batch_iter, "set_postfix") and (
                batch_idx % TQDM_UPDATE_EVERY == 0 or batch_idx == len
            ):
                batch_iter.set_postfix(batch_loss=f"{loss.item():.4f}"

        avg = running / max(1, count)
        entry = {"epoch": epoch + 1, "avg_loss": float(avg)}

        should_track = track_metrics and (((epoch + 1) % track_metric_
        if should_track:
            metrics = evaluate_metric_bundle(
                student,
                device=device,
                straightness_batch_size=EVAL_BATCH_SIZE,
                straightness_pair_steps=PAIR_STEPS,
                straightness_pair_solver=PAIR_SOLVER,
                straightness_batches=track_straightness_batches,
                endpoint_batch_size=EVAL_BATCH_SIZE,
                endpoint_steps=track_endpoint_steps,
                endpoint_solver=SAMPLE_SOLVER,
                endpoint_batches=track_endpoint_batches,
            )
            entry.update(metrics)
            print(
                f"[{stage_name}] epoch {epoch + 1}/{epochs} - loss: {a
                f"straightness: {metrics['paper_straightness']:.6f} |
                f"endpoint_consistency: {metrics['endpoint_consistency
                f"quadratic_transport_cost: {metrics['quadratic_transp
            )
        else:
```

```
            print(f"[{stage_name}] epoch {epoch + 1}/{epochs} - loss:

        history.append(entry)

        if checkpoint_dir is not None:
            torch.save(
                {
                    "model": student.state_dict(),
                    "losses": losses,
                    "history": history,
                    "epoch": epoch + 1,
                    "stage_name": stage_name,
                },
                checkpoint_dir / f"{stage_name}_epoch_{epoch + 1:02d}.
            )

    return student, losses, history
```

Block 6A — One-Step Distillation Helpers

These helpers implement the paper's **final one-step distillation idea**.

After we obtain a (k)-rectified flow, we can train a direct one-step predictor [\hat T(z_0)=z_0+v_ heta(z_0,0)] to map the starting noise (z_0) directly to the final endpoint (Z_1^k).

For this section, the right metric is **one-step endpoint MSE**, because the distilled model no longer defines a full trajectory.
The **straightness metric still belongs to the source rectified flow**, and it explains why this one-step approximation becomes easier after rectification.

In [ ]:
```
# Block 6A: One-step distillation helpers

@torch.no_grad()
def one_step_predict(model: nn.Module, x0: torch.Tensor) -> torch.Tens
    model.eval()
    t0 = torch.zeros(x0.shape[0], 1, 1, 1, device=x0.device, dtype=x0.
    return x0 + model(x0, t0)


@torch.no_grad()
def sample_images_one_step(
    model: nn.Module,
    n_samples: int,
    batch_size: int,
    device: torch.device,
):
    outs = []
    remaining = n_samples
    while remaining > 0:
```

```
        cur = min(batch_size, remaining)
        x0 = torch.randn(cur, 1, 28, 28, device=device)
        x1_hat = one_step_predict(model, x0)
        outs.append(x1_hat.cpu())
        remaining -= cur
    return torch.cat(outs, dim=0)


@torch.no_grad()
def estimate_t0_endpoint_consistency(
    flow_model: nn.Module,
    device: torch.device,
    batch_size: int = 128,
    pair_steps: int = 32,
    pair_solver: str = "midpoint",
    batches: int = 8,
):
    flow_model.eval()
    vals = []
    for _ in range(batches):
        x0 = torch.randn(batch_size, 1, 28, 28, device=device)
        z1 = integrate_flow(flow_model, x0, num_steps=pair_steps, solv
        z1_hat = one_step_predict(flow_model, x0)
        vals.append(F.mse_loss(z1_hat, z1).item())
    return float(np.mean(vals))


@torch.no_grad()
def estimate_distilled_one_step_mse(
    distilled_model: nn.Module,
    source_flow: nn.Module,
    device: torch.device,
    batch_size: int = 128,
    pair_steps: int = 32,
    pair_solver: str = "midpoint",
    batches: int = 8,
):
    distilled_model.eval()
    source_flow.eval()
    vals = []
    for _ in range(batches):
        x0 = torch.randn(batch_size, 1, 28, 28, device=device)
        z1 = integrate_flow(source_flow, x0, num_steps=pair_steps, sol
        z1_hat = one_step_predict(distilled_model, x0)
        vals.append(F.mse_loss(z1_hat, z1).item())
    return float(np.mean(vals))


def train_distillation_stage(
    source_flow: nn.Module,
    loader: DataLoader,
    device: torch.device,
```

```
    epochs: int,
    lr: float,
    stage_name: str,
    pair_steps: int,
    pair_solver: str,
    track_metrics: bool = True,
    track_metric_every: int = 1,
    checkpoint_dir: Path | None = None,
):
    student = copy.deepcopy(source_flow).to(device)
    source_flow.eval()
    student.train()

    optimizer = AdamW(student.parameters(), lr=lr, betas=(0.9, 0.99))
    losses = []
    history = []

    if checkpoint_dir is not None:
        checkpoint_dir.mkdir(parents=True, exist_ok=True)

    batch_size = loader.batch_size or 128
    for epoch in range(epochs):
        running = 0.0
        count = 0
        batch_iter = maybe_tqdm(
            loader,
            enabled=SHOW_TQDM,
            desc=f"{stage_name} | epoch {epoch + 1}/{epochs}",
            leave=False,
            dynamic_ncols=True,
        )
        for batch_idx, (_x, _) in enumerate(batch_iter, start=1):
            x0 = torch.randn(batch_size, 1, 28, 28, device=device)
            with torch.no_grad():
                z1 = integrate_flow(source_flow, x0, num_steps=pair_st

            t0 = torch.zeros(batch_size, 1, 1, 1, device=device)
            target_disp = z1 - x0

            optimizer.zero_grad(set_to_none=True)
            pred_disp = student(x0, t0)
            loss = F.mse_loss(pred_disp, target_disp)
            loss.backward()
            optimizer.step()

            losses.append(loss.item())
            running += loss.item()
            count += 1

            if SHOW_TQDM and hasattr(batch_iter, "set_postfix") and (
                batch_idx % TQDM_UPDATE_EVERY == 0 or batch_idx == len
            ):
```

```
                batch_iter.set_postfix(batch_loss=f"{loss.item():.4f}"

        avg = running / max(1, count)
        entry = {"epoch": epoch + 1, "avg_loss": float(avg)}

        should_track = track_metrics and (((epoch + 1) % track_metric_
        if should_track:
            one_step_mse = estimate_distilled_one_step_mse(
                distilled_model=student,
                source_flow=source_flow,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
                pair_steps=PAIR_STEPS,
                pair_solver=PAIR_SOLVER,
                batches=max(2, TRACK_STRAIGHTNESS_BATCHES),
            )
            entry["one_step_endpoint_mse"] = one_step_mse
            print(
                f"[{stage_name}] epoch {epoch + 1}/{epochs} - loss: {a
                f"one_step_endpoint_mse: {one_step_mse:.6f}"
            )
        else:
            print(f"[{stage_name}] epoch {epoch + 1}/{epochs} - loss:

        history.append(entry)

        if checkpoint_dir is not None:
            torch.save(
                {
                    "model": student.state_dict(),
                    "losses": losses,
                    "history": history,
                    "epoch": epoch + 1,
                    "stage_name": stage_name,
                },
                checkpoint_dir / f"{stage_name}_epoch_{epoch + 1:02d}.
            )

    return student, losses, history
```

Main Experiments: Stage 1 and Stage 2

1. **sample grids across NFEs**,
2. **straightness**,
3. **endpoint-consistency**,
4. **quadratic transport cost**,
5. **one-step distillation** after (k=2).

The optional (k=3) extension appears only after the main results above.

Block 7 — Train Stage 1 (1-Rectified Flow / Standard CFM)

This is the baseline training stage. We learn a velocity field on MNIST using the standard conditional flow matching objective before applying any rectification.

In [ ]:
```
# Block 7: Train stage 1 (1-Rectified Flow / standard CFM stage)

stage1_ckpt_dir = OUT_DIR / "stage1_checkpoints"

stage1_model, stage1_losses, stage1_history = train_cfm_stage(
    model=teacher,
    loader=train_loader,
    device=device,
    epochs=CFM_EPOCHS,
    lr=LR,
    stage_name="stage1_cfm",
    track_metrics=TRACK_METRICS,
    track_metric_every=TRACK_METRIC_EVERY,
    track_straightness_batches=TRACK_STRAIGHTNESS_BATCHES,
    track_endpoint_batches=TRACK_ENDPOINT_BATCHES,
    track_endpoint_steps=TRACK_ENDPOINT_STEPS,
    checkpoint_dir=stage1_ckpt_dir if SAVE_EPOCH_CHECKPOINTS else None
)

torch.save(
    {"model": stage1_model.state_dict(), "losses": stage1_losses, "his
    OUT_DIR / "stage1_cfm.pt",
)
with open(OUT_DIR / "stage1_history.json", "w", encoding="utf-8") as f
    json.dump(stage1_history, f, indent=2)

print("Saved:", OUT_DIR / "stage1_cfm.pt")
print("Saved:", OUT_DIR / "stage1_history.json")
```

```
[stage1_cfm] epoch 1/50 - loss: 0.197335
```

```
[stage1_cfm] epoch 2/50 - loss: 0.127747 | straightness: 0.038754 | end
point_consistency: 0.015736 | quadratic_transport_cost: 737.762075
```

```
[stage1_cfm] epoch 3/50 - loss: 0.118965
```

```
[stage1_cfm] epoch 4/50 - loss: 0.115760 | straightness: 0.042031 | end
point_consistency: 0.018519 | quadratic_transport_cost: 741.233531
```

```
[stage1_cfm] epoch 5/50 - loss: 0.112378
```

```
[stage1_cfm] epoch 6/50 - loss: 0.110484 | straightness: 0.047571 | end
point_consistency: 0.020815 | quadratic_transport_cost: 740.556864
```

```
[stage1_cfm] epoch 7/50 - loss: 0.108757
```

```
[stage1_cfm] epoch 8/50 - loss: 0.107389 | straightness: 0.042271 | end
point_consistency: 0.018655 | quadratic_transport_cost: 749.629283
```

```
[stage1_cfm] epoch 9/50 - loss: 0.106037
```

```
[stage1_cfm] epoch 10/50 - loss: 0.104886 | straightness: 0.042938 | en
dpoint_consistency: 0.018973 | quadratic_transport_cost: 749.304545
```

```
[stage1_cfm] epoch 11/50 - loss: 0.103896
```

```
[stage1_cfm] epoch 12/50 - loss: 0.103314 | straightness: 0.046659 | en
dpoint_consistency: 0.018821 | quadratic_transport_cost: 753.073619
```

```
[stage1_cfm] epoch 13/50 - loss: 0.102452
```

```
[stage1_cfm] epoch 14/50 - loss: 0.101715 | straightness: 0.039964 | en
dpoint_consistency: 0.017799 | quadratic_transport_cost: 751.191833

[stage1_cfm] epoch 15/50 - loss: 0.101526

[stage1_cfm] epoch 16/50 - loss: 0.100837 | straightness: 0.048228 | en
dpoint_consistency: 0.019754 | quadratic_transport_cost: 760.723816

[stage1_cfm] epoch 17/50 - loss: 0.100424

[stage1_cfm] epoch 18/50 - loss: 0.100299 | straightness: 0.042670 | en
dpoint_consistency: 0.019688 | quadratic_transport_cost: 755.777791

[stage1_cfm] epoch 19/50 - loss: 0.099833

[stage1_cfm] epoch 20/50 - loss: 0.099082 | straightness: 0.042833 | en
dpoint_consistency: 0.018902 | quadratic_transport_cost: 756.212636

[stage1_cfm] epoch 21/50 - loss: 0.099350

[stage1_cfm] epoch 22/50 - loss: 0.098739 | straightness: 0.049629 | en
dpoint_consistency: 0.020391 | quadratic_transport_cost: 756.302144

[stage1_cfm] epoch 23/50 - loss: 0.098470
```

```
[stage1_cfm] epoch 24/50 - loss: 0.098120 | straightness: 0.037171 | en
dpoint_consistency: 0.017733 | quadratic_transport_cost: 755.620026

[stage1_cfm] epoch 25/50 - loss: 0.097674

[stage1_cfm] epoch 26/50 - loss: 0.097571 | straightness: 0.039066 | en
dpoint_consistency: 0.018956 | quadratic_transport_cost: 763.451589

[stage1_cfm] epoch 27/50 - loss: 0.097474

[stage1_cfm] epoch 28/50 - loss: 0.096823 | straightness: 0.046904 | en
dpoint_consistency: 0.018760 | quadratic_transport_cost: 759.991211

[stage1_cfm] epoch 29/50 - loss: 0.096826

[stage1_cfm] epoch 30/50 - loss: 0.096179 | straightness: 0.038498 | en
dpoint_consistency: 0.019320 | quadratic_transport_cost: 762.034292

[stage1_cfm] epoch 31/50 - loss: 0.096418

[stage1_cfm] epoch 32/50 - loss: 0.096124 | straightness: 0.041121 | en
dpoint_consistency: 0.018861 | quadratic_transport_cost: 762.425262

[stage1_cfm] epoch 33/50 - loss: 0.095868
```

```
[stage1_cfm] epoch 34/50 - loss: 0.095880 | straightness: 0.036776 | en
dpoint_consistency: 0.018850 | quadratic_transport_cost: 761.411570

[stage1_cfm] epoch 35/50 - loss: 0.095962

[stage1_cfm] epoch 36/50 - loss: 0.095679 | straightness: 0.038738 | en
dpoint_consistency: 0.018517 | quadratic_transport_cost: 756.123545

[stage1_cfm] epoch 37/50 - loss: 0.095558

[stage1_cfm] epoch 38/50 - loss: 0.095173 | straightness: 0.041369 | en
dpoint_consistency: 0.019131 | quadratic_transport_cost: 756.690633

[stage1_cfm] epoch 39/50 - loss: 0.094857

[stage1_cfm] epoch 40/50 - loss: 0.094719 | straightness: 0.035409 | en
dpoint_consistency: 0.016655 | quadratic_transport_cost: 757.036875

[stage1_cfm] epoch 41/50 - loss: 0.094778

[stage1_cfm] epoch 42/50 - loss: 0.094829 | straightness: 0.044691 | en
dpoint_consistency: 0.018262 | quadratic_transport_cost: 763.608897

[stage1_cfm] epoch 43/50 - loss: 0.094561
```

```
[stage1_cfm] epoch 44/50 - loss: 0.094523 | straightness: 0.038204 | en
dpoint_consistency: 0.018736 | quadratic_transport_cost: 759.324768
```

```
[stage1_cfm] epoch 45/50 - loss: 0.093976
```

```
[stage1_cfm] epoch 46/50 - loss: 0.094163 | straightness: 0.038916 | en
dpoint_consistency: 0.018443 | quadratic_transport_cost: 757.253611
```

```
[stage1_cfm] epoch 47/50 - loss: 0.094236
```

```
[stage1_cfm] epoch 48/50 - loss: 0.094218 | straightness: 0.036550 | en
dpoint_consistency: 0.018643 | quadratic_transport_cost: 757.713308
```

```
[stage1_cfm] epoch 49/50 - loss: 0.093709
```

```
[stage1_cfm] epoch 50/50 - loss: 0.093762 | straightness: 0.035552 | en
dpoint_consistency: 0.017661 | quadratic_transport_cost: 760.282176
Saved: mnist_rectified_flow_notebook_runs/stage1_cfm.pt
Saved: mnist_rectified_flow_notebook_runs/stage1_history.json
```

Block 8 — Train Stage 2 (2-Rectified Flow via Reflow)

This cell performs the main algorithmic step from the paper: reflow. It generates endpoint pairs using the trained stage-1 model and then trains a new model on those pairs, which should produce a straighter flow.

In [ ]:
```
# Block 9: Train stage 2 (2-Rectified Flow via reflow)

stage2_ckpt_dir = OUT_DIR / "stage2_checkpoints"

stage2_model, stage2_losses, stage2_history = train_reflow_stage(
```

```
    teacher=stage1_model,
    loader=train_loader,
    device=device,
    epochs=REFLOW_EPOCHS,
    lr=LR,
    stage_name="stage2_reflow",
    pair_steps=PAIR_STEPS,
    pair_solver=PAIR_SOLVER,
    track_metrics=TRACK_METRICS,
    track_metric_every=TRACK_METRIC_EVERY,
    track_straightness_batches=TRACK_STRAIGHTNESS_BATCHES,
    track_endpoint_batches=TRACK_ENDPOINT_BATCHES,
    track_endpoint_steps=TRACK_ENDPOINT_STEPS,
    checkpoint_dir=stage2_ckpt_dir if SAVE_EPOCH_CHECKPOINTS else None
)

torch.save(
    {"model": stage2_model.state_dict(), "losses": stage2_losses, "his
    OUT_DIR / "stage2_reflow.pt",
)
with open(OUT_DIR / "stage2_history.json", "w", encoding="utf-8") as f
    json.dump(stage2_history, f, indent=2)

print("Saved:", OUT_DIR / "stage2_reflow.pt")
print("Saved:", OUT_DIR / "stage2_history.json")
```

```
[stage2_reflow] epoch 1/20 - loss: 0.009780
```

```
[stage2_reflow] epoch 2/20 - loss: 0.007861 | straightness: 0.006643 |
endpoint_consistency: 0.002399 | quadratic_transport_cost: 754.824814
```

```
[stage2_reflow] epoch 3/20 - loss: 0.007553
```

```
[stage2_reflow] epoch 4/20 - loss: 0.007455 | straightness: 0.003574 |
endpoint_consistency: 0.001915 | quadratic_transport_cost: 751.009140
```

```
[stage2_reflow] epoch 5/20 - loss: 0.007040
```

```
[stage2_reflow] epoch 6/20 - loss: 0.006770 | straightness: 0.006848 |
endpoint_consistency: 0.001952 | quadratic_transport_cost: 752.255493
```

```
[stage2_reflow] epoch 7/20 - loss: 0.006815
```

```
[stage2_reflow] epoch 8/20 - loss: 0.006532 | straightness: 0.006643 |
endpoint_consistency: 0.001822 | quadratic_transport_cost: 751.294266
```

```
[stage2_reflow] epoch 9/20 - loss: 0.006499
```

```
[stage2_reflow] epoch 10/20 - loss: 0.006260 | straightness: 0.004488 |
endpoint_consistency: 0.001686 | quadratic_transport_cost: 753.563980
```

```
[stage2_reflow] epoch 11/20 - loss: 0.006434
```

```
[stage2_reflow] epoch 12/20 - loss: 0.006126 | straightness: 0.003349 |
endpoint_consistency: 0.001626 | quadratic_transport_cost: 751.156158
```

```
[stage2_reflow] epoch 13/20 - loss: 0.005988
```

```
[stage2_reflow] epoch 14/20 - loss: 0.005935 | straightness: 0.002759 |
endpoint_consistency: 0.001591 | quadratic_transport_cost: 747.320496
```

```
[stage2_reflow] epoch 15/20 - loss: 0.006336
```

```
[stage2_reflow] epoch 16/20 - loss: 0.006051 | straightness: 0.002609 |
endpoint_consistency: 0.001574 | quadratic_transport_cost: 751.438019
```

```
[stage2_reflow] epoch 17/20 - loss: 0.006066
```

```
[stage2_reflow] epoch 18/20 - loss: 0.005811 | straightness: 0.005462 |
endpoint_consistency: 0.001534 | quadratic_transport_cost: 756.148987
```

```
[stage2_reflow] epoch 19/20 - loss: 0.005553
```

```
[stage2_reflow] epoch 20/20 - loss: 0.005945 | straightness: 0.002922 |
endpoint_consistency: 0.001464 | quadratic_transport_cost: 752.895767
Saved: mnist_rectified_flow_notebook_runs/stage2_reflow.pt
Saved: mnist_rectified_flow_notebook_runs/stage2_history.json
```

Block 9 — Main Qualitative Experiment: Sample Grids Across NFEs

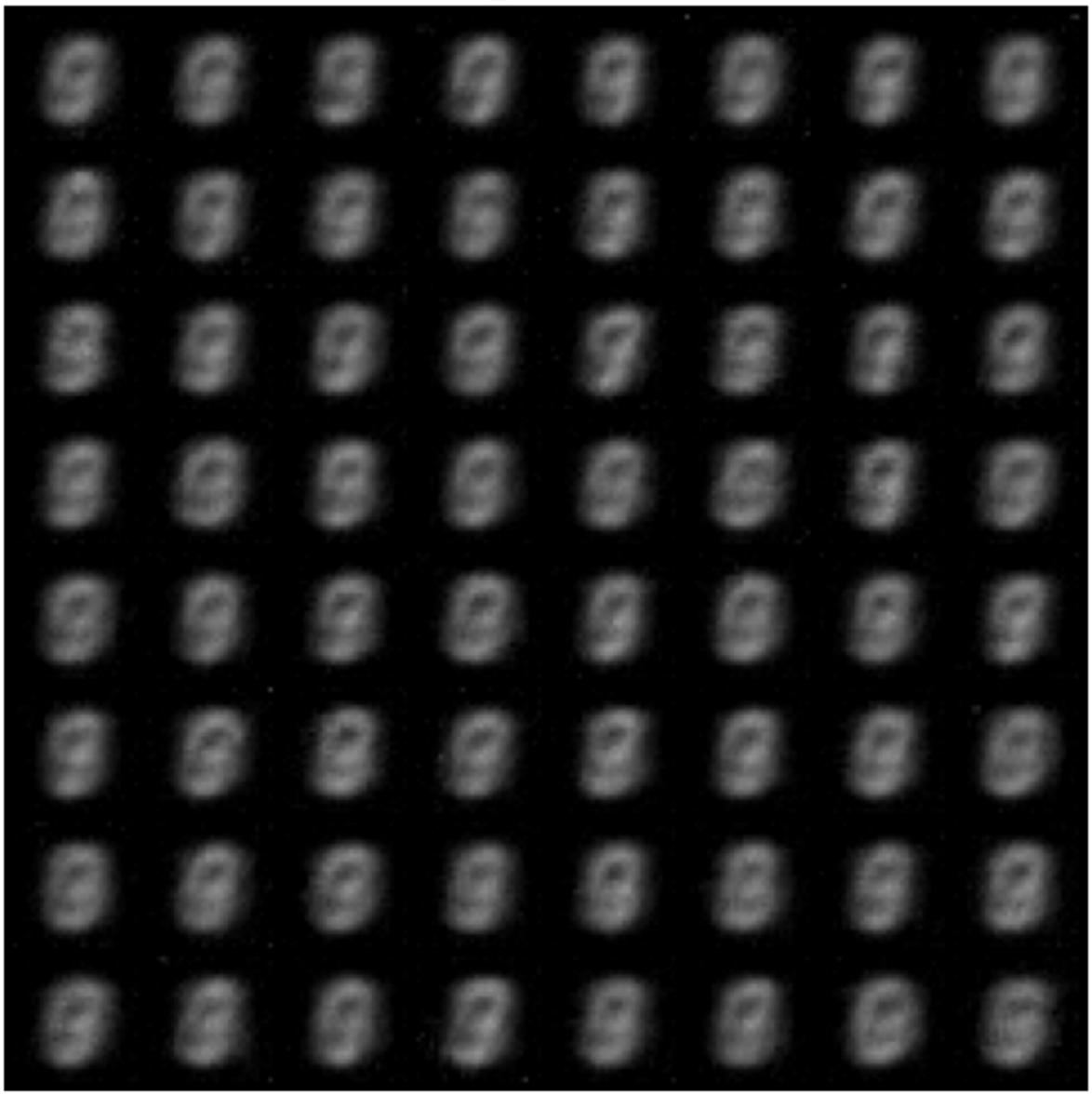
This cell generates unconditional MNIST samples for several numbers of function evaluations (NFEs), such as 1, 2, 4, 8, and 16. These grids provide the main visual comparison between the baseline flow and the rectified flow.

In [ ]:
```python
# Block 11: Main qualitative experiment
# Compare stage 1 vs stage 2 at NFE = 1, 2, 4, 8, 16

main_stage_models = {
    "stage1_cfm": stage1_model.eval(),
    "stage2_reflow": stage2_model.eval(),
}

for stage_name, model in main_stage_models.items():
    stage_dir = OUT_DIR / stage_name
    stage_dir.mkdir(parents=True, exist_ok=True)
    for steps in SAMPLE_STEPS:
        samples = sample_images(
            model=model,
            n_samples=VIS_N,
            batch_size=VIS_N,
```

```
            num_steps=steps,
            device=device,
            solver=SAMPLE_SOLVER,
        )
        save_sample_grid(
            samples,
            stage_dir / f"samples_{steps:02d}_steps.png",
            show_inline=SHOW_SAVED_SAMPLE_GRIDS and (steps in INLINE_S
            title=f"{stage_name} | NFE={steps}",
        )
```

```
Saved: mnist_rectified_flow_notebook_runs/stage1_cfm/samples_01_steps.p
ng
```

stage1_cfm | NFE=1



```
Saved: mnist_rectified_flow_notebook_runs/stage1_cfm/samples_02_steps.p
ng
```

stage1_cfm | NFE=2

```
Saved: mnist_rectified_flow_notebook_runs/stage1_cfm/samples_04_steps.p
ng
```

stage1_cfm | NFE=4

```
Saved: mnist_rectified_flow_notebook_runs/stage1_cfm/samples_08_steps.p
ng
Saved: mnist_rectified_flow_notebook_runs/stage1_cfm/samples_16_steps.p
ng
```

stage1_cfm | NFE=16

```
Saved: mnist_rectified_flow_notebook_runs/stage2_reflow/samples_01_step
s.png
```

stage2_reflow | NFE=1

```
Saved: mnist_rectified_flow_notebook_runs/stage2_reflow/samples_02_step
s.png
```

stage2_reflow | NFE=2

```
Saved: mnist_rectified_flow_notebook_runs/stage2_reflow/samples_04_step
s.png
```

stage2_reflow | NFE=4

```
Saved: mnist_rectified_flow_notebook_runs/stage2_reflow/samples_08_step
s.png
Saved: mnist_rectified_flow_notebook_runs/stage2_reflow/samples_16_step
s.png
```

Block 12 — Main Quantitative Comparison for Stage 1 vs Stage 2

This cell evaluates both models with the three flow-side metrics and stores the results in a compact dictionary:

- paper-style **straightness**,
- **endpoint-consistency**,
- **quadratic transport cost**.

In [ ]:
```
# Block 12: Main quantitative table for stage 1 vs stage 2

main_results = {}
for stage_name, model in main_stage_models.items():
    main_results[stage_name] = {
        "paper_straightness": estimate_paper_straightness(
```

```
            model,
            device=device,
            batch_size=EVAL_BATCH_SIZE,
            pair_steps=PAIR_STEPS,
            pair_solver=PAIR_SOLVER,
            batches=FINAL_METRIC_BATCHES,
        ),
        "endpoint_consistency": estimate_endpoint_consistency(
            model,
            device=device,
            batch_size=EVAL_BATCH_SIZE,
            num_steps=max(SAMPLE_STEPS),
            solver=SAMPLE_SOLVER,
            batches=max(2, FINAL_METRIC_BATCHES // 2),
        ),
        "quadratic_transport_cost": estimate_quadratic_transport_cost(
            model,
            device=device,
            batch_size=EVAL_BATCH_SIZE,
            pair_steps=PAIR_STEPS,
            pair_solver=PAIR_SOLVER,
            batches=FINAL_METRIC_BATCHES,
        ),
    }

print(json.dumps(main_results, indent=2))

with open(OUT_DIR / "main_metrics_stage1_stage2.json", "w", encoding="
    json.dump(main_results, f, indent=2)

print("Saved:", OUT_DIR / "main_metrics_stage1_stage2.json")
```

```
{
  "stage1_cfm": {
    "paper_straightness": 0.03651069179177284,
    "endpoint_consistency": 0.01778524607161671,
    "quadratic_transport_cost": 760.6466125488281
  },
  "stage2_reflow": {
    "paper_straightness": 0.0025914738420397044,
    "endpoint_consistency": 0.0014656951692160192,
    "quadratic_transport_cost": 752.8458068847656
  }
}
Saved: mnist_rectified_flow_notebook_runs/main_metrics_stage1_stage2.js
on
```

Block 13 — Compact Metric Plots for Stage 1 vs Stage 2

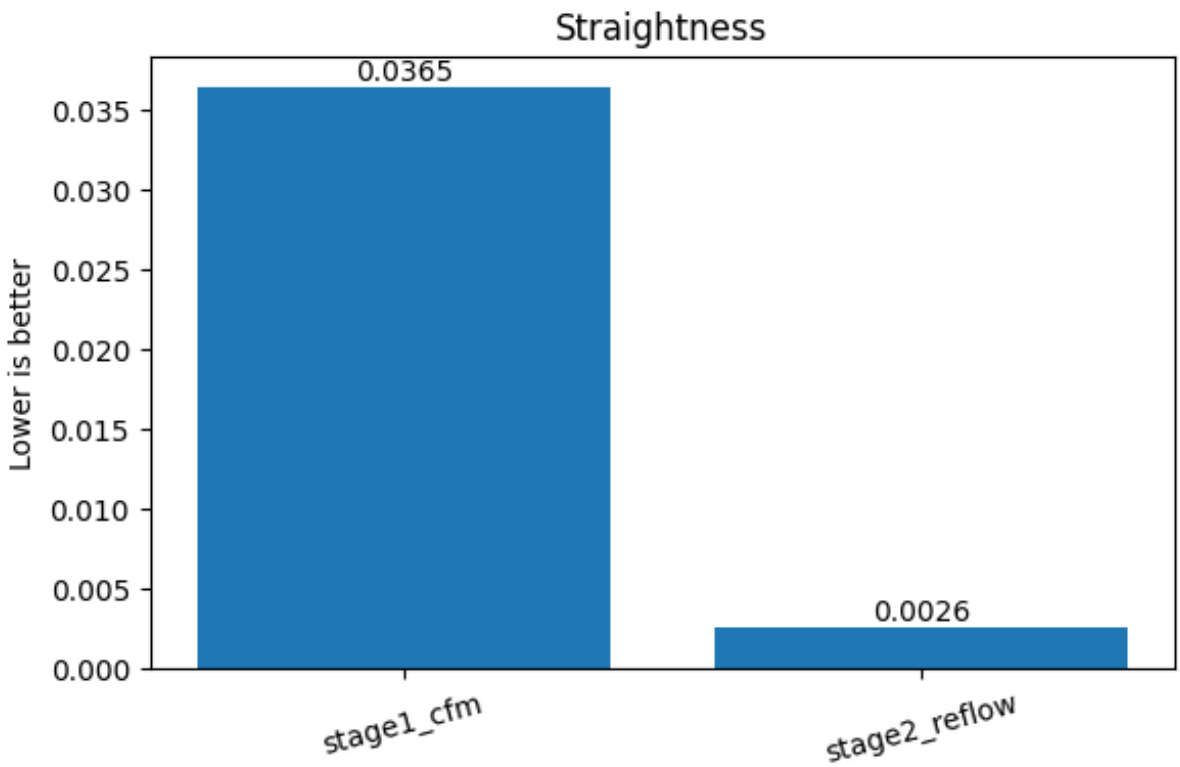
This cell plots the final values of the three main flow-side diagnostics across the two main stages:

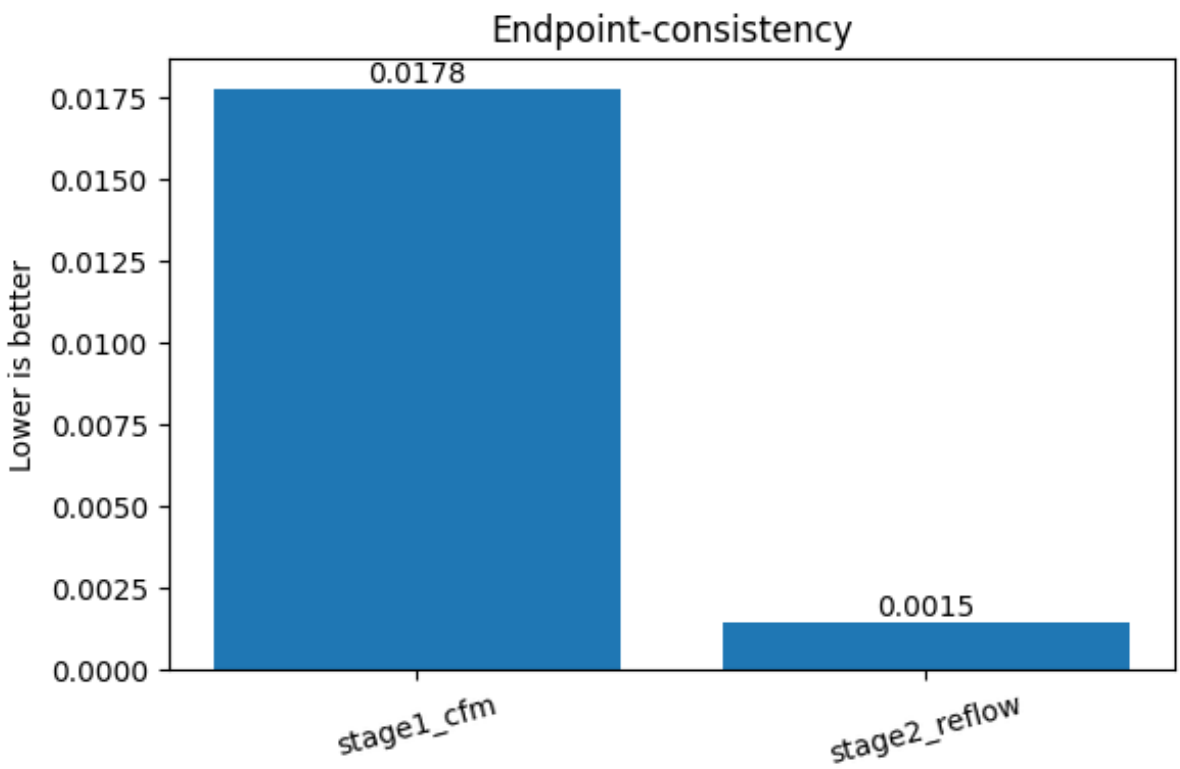
- paper-style **straightness**,
- **endpoint-consistency**,
- **quadratic transport cost**.

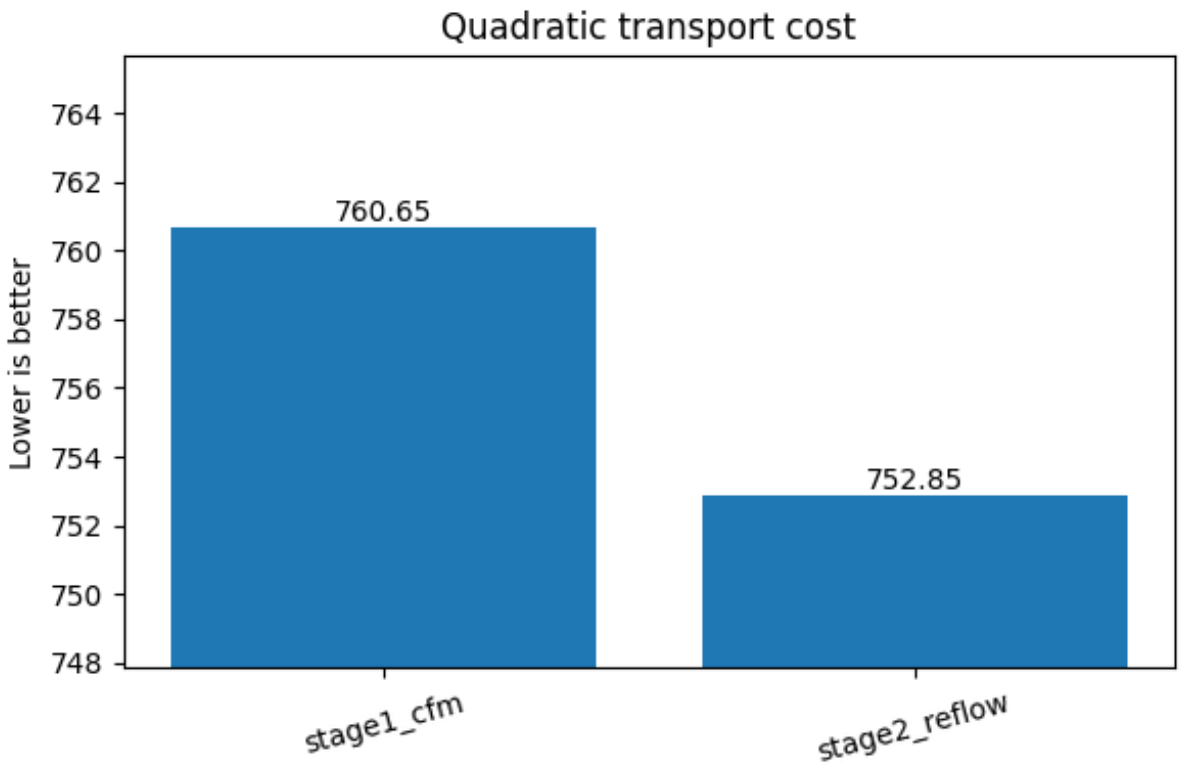
These plots are meant to support the same story as the sample grids: stage 2 should ideally be straighter, have better local-to-endpoint alignment, and not increase the chosen quadratic transport cost.

In [ ]:
```
# Block 13: Compact metric plots for stage 1 vs stage 2

stage_names = list(main_results.keys())
straightness_vals = [main_results[s]["paper_straightness"] for s in st
endpoint_vals = [main_results[s]["endpoint_consistency"] for s in stag
transport_vals = [main_results[s]["quadratic_transport_cost"] for s in

# Paper straightness
plt.figure(figsize=(6, 4))
bars = plt.bar(stage_names, straightness_vals)
plt.title("Straightness")
plt.ylabel("Lower is better")
plt.xticks(rotation=15)
for b, v in zip(bars, straightness_vals):
    plt.text(b.get_x() + b.get_width()/2, v, f"{v:.4f}", ha="center",
plt.tight_layout()
plt.show()

# Endpoint-consistency
plt.figure(figsize=(6, 4))
bars = plt.bar(stage_names, endpoint_vals)
plt.title("Endpoint-consistency")
plt.ylabel("Lower is better")
plt.xticks(rotation=15)
for b, v in zip(bars, endpoint_vals):
    plt.text(b.get_x() + b.get_width()/2, v, f"{v:.4f}", ha="center",
plt.tight_layout()
plt.show()

# Quadratic transport cost
plt.figure(figsize=(6, 4))
bars = plt.bar(stage_names, transport_vals)
plt.title("Quadratic transport cost")
plt.ylabel("Lower is better")
plt.xticks(rotation=15)

ymin = min(transport_vals) - 5
ymax = max(transport_vals) + 5
plt.ylim(ymin, ymax)

for b, v in zip(bars, transport_vals):
    plt.text(b.get_x() + b.get_width()/2, v, f"{v:.2f}", ha="center",
```

```
plt.tight_layout()
plt.show()
```







Checkpoint Metric Curves

These are the progress plots that matter most for this notebook: how the paper-style **straightness**, the extra **endpoint-consistency**, and the **quadratic transport cost** evolve during training.

Block 14 — Checkpoint Metric Histories

This cell prints the metric history recorded during training for stage 1 and stage 2. These histories are useful for checking whether rectification is improving the desired geometry progressively over epochs.

In [ ]:
```
# Block 14: View the epoch-wise metric histories for stage 1 and stage

print("Stage 1 history:")
print(json.dumps(stage1_history, indent=2))

print("\nStage 2 history:")
print(json.dumps(stage2_history, indent=2))
```

```
Stage 1 history:
[
  {
    "epoch": 1,
    "avg_loss": 0.19733538838405895
  },
```

```
{
  "epoch": 2,
  "avg_loss": 0.1277466711994165,
  "paper_straightness": 0.03875364611546198,
  "endpoint_consistency": 0.01573610460013697,
  "quadratic_transport_cost": 737.7620747884115
},
{
  "epoch": 3,
  "avg_loss": 0.11896544252323289
},
{
  "epoch": 4,
  "avg_loss": 0.11575967267665088,
  "paper_straightness": 0.042031156520048775,
  "endpoint_consistency": 0.018519386858997677,
  "quadratic_transport_cost": 741.2335306803385
},
{
  "epoch": 5,
  "avg_loss": 0.11237780504629143
},
{
  "epoch": 6,
  "avg_loss": 0.11048422753810883,
  "paper_straightness": 0.047571210811535515,
  "endpoint_consistency": 0.02081481345476277,
  "quadratic_transport_cost": 740.5568644205729
},
{
  "epoch": 7,
  "avg_loss": 0.10875665501524241
},
{
  "epoch": 8,
  "avg_loss": 0.10738923024927449,
  "paper_straightness": 0.04227110246817271,
  "endpoint_consistency": 0.018655129831586237,
  "quadratic_transport_cost": 749.6292826334635
},
{
  "epoch": 9,
  "avg_loss": 0.10603666756079237
},
{
  "epoch": 10,
  "avg_loss": 0.1048857729531761,
  "paper_straightness": 0.04293780649701754,
  "endpoint_consistency": 0.018973183210732714,
  "quadratic_transport_cost": 749.3045450846354
},
{
```

```
    "epoch": 11,
    "avg_loss": 0.10389580107970625
  },
  {
    "epoch": 12,
    "avg_loss": 0.10331365074484776,
    "paper_straightness": 0.04665905920167764,
    "endpoint_consistency": 0.01882067408439525,
    "quadratic_transport_cost": 753.0736185709635
  },
  {
    "epoch": 13,
    "avg_loss": 0.10245220309011957
  },
  {
    "epoch": 14,
    "avg_loss": 0.10171484674971837,
    "paper_straightness": 0.03996360984941324,
    "endpoint_consistency": 0.017798827170508957,
    "quadratic_transport_cost": 751.1918334960938
  },
  {
    "epoch": 15,
    "avg_loss": 0.10152647499408987
  },
  {
    "epoch": 16,
    "avg_loss": 0.1008372012939718,
    "paper_straightness": 0.04822824212412039,
    "endpoint_consistency": 0.019753912462533663,
    "quadratic_transport_cost": 760.7238159179688
  },
  {
    "epoch": 17,
    "avg_loss": 0.10042418660516413
  },
  {
    "epoch": 18,
    "avg_loss": 0.10029899475411472,
    "paper_straightness": 0.04266980662941933,
    "endpoint_consistency": 0.01968751953802439,
    "quadratic_transport_cost": 755.7777913411459
  },
  {
    "epoch": 19,
    "avg_loss": 0.0998331680893898
  },
  {
    "epoch": 20,
    "avg_loss": 0.09908168127712531,
    "paper_straightness": 0.042832693085074425,
    "endpoint_consistency": 0.018902206182360715,
```

```
    "quadratic_transport_cost": 756.212636311849
  },
  {
    "epoch": 21,
    "avg_loss": 0.09935017387008566
  },
  {
    "epoch": 22,
    "avg_loss": 0.09873926983429836,
    "paper_straightness": 0.049628653253118195,
    "endpoint_consistency": 0.020390586892062856,
    "quadratic_transport_cost": 756.3021443684896
  },
  {
    "epoch": 23,
    "avg_loss": 0.09846995384074174
  },
  {
    "epoch": 24,
    "avg_loss": 0.09811997391347192,
    "paper_straightness": 0.037170641124248505,
    "endpoint_consistency": 0.017733182703182138,
    "quadratic_transport_cost": 755.6200256347656
  },
  {
    "epoch": 25,
    "avg_loss": 0.09767378545087627
  },
  {
    "epoch": 26,
    "avg_loss": 0.09757125000349987,
    "paper_straightness": 0.039065919195612274,
    "endpoint_consistency": 0.01895613406099983,
    "quadratic_transport_cost": 763.4515889485677
  },
  {
    "epoch": 27,
    "avg_loss": 0.09747412788052844
  },
  {
    "epoch": 28,
    "avg_loss": 0.09682311340529695,
    "paper_straightness": 0.04690366176267465,
    "endpoint_consistency": 0.01875991346162209,
    "quadratic_transport_cost": 759.9912109375
  },
  {
    "epoch": 29,
    "avg_loss": 0.0968262854263059
  },
  {
    "epoch": 30,
```

```
    "avg_loss": 0.09617903314403489,
    "paper_straightness": 0.03849810175597668,
    "endpoint_consistency": 0.019320278473969665,
    "quadratic_transport_cost": 762.0342915852865
  },
  {
    "epoch": 31,
    "avg_loss": 0.09641762023680231
  },
  {
    "epoch": 32,
    "avg_loss": 0.09612441190287598,
    "paper_straightness": 0.04112059250473976,
    "endpoint_consistency": 0.018860622956481166,
    "quadratic_transport_cost": 762.4252624511719
  },
  {
    "epoch": 33,
    "avg_loss": 0.09586808357674342
  },
  {
    "epoch": 34,
    "avg_loss": 0.09588031701616243,
    "paper_straightness": 0.03677615337073803,
    "endpoint_consistency": 0.01885046607935692,
    "quadratic_transport_cost": 761.4115702311198
  },
  {
    "epoch": 35,
    "avg_loss": 0.09596213693611133
  },
  {
    "epoch": 36,
    "avg_loss": 0.09567940561498842,
    "paper_straightness": 0.038737982511520386,
    "endpoint_consistency": 0.01851697408835662,
    "quadratic_transport_cost": 756.123545328776
  },
  {
    "epoch": 37,
    "avg_loss": 0.09555821798932858
  },
  {
    "epoch": 38,
    "avg_loss": 0.09517323701745933,
    "paper_straightness": 0.041368912905454636,
    "endpoint_consistency": 0.019130685801201253,
    "quadratic_transport_cost": 756.6906331380209
  },
  {
    "epoch": 39,
    "avg_loss": 0.09485695597070914
```

```
    },
    {
      "epoch": 40,
      "avg_loss": 0.09471876105755313,
      "paper_straightness": 0.035408928990364075,
      "endpoint_consistency": 0.016655497403917252,
      "quadratic_transport_cost": 757.036875406901
    },
    {
      "epoch": 41,
      "avg_loss": 0.0947778273660403
    },
    {
      "epoch": 42,
      "avg_loss": 0.09482884685644227,
      "paper_straightness": 0.044691137969493866,
      "endpoint_consistency": 0.01826225903039358,
      "quadratic_transport_cost": 763.608896891276
    },
    {
      "epoch": 43,
      "avg_loss": 0.09456085276782003
    },
    {
      "epoch": 44,
      "avg_loss": 0.0945234182330533,
      "paper_straightness": 0.03820387149850527,
      "endpoint_consistency": 0.01873563107744758,
      "quadratic_transport_cost": 759.3247680664062
    },
    {
      "epoch": 45,
      "avg_loss": 0.09397609057462114
    },
    {
      "epoch": 46,
      "avg_loss": 0.09416337484796332,
      "paper_straightness": 0.03891610478361448,
      "endpoint_consistency": 0.01844329735001793,
      "quadratic_transport_cost": 757.2536112467448
    },
    {
      "epoch": 47,
      "avg_loss": 0.09423572350388919
    },
    {
      "epoch": 48,
      "avg_loss": 0.0942179109646469,
      "paper_straightness": 0.03654965634147326,
      "endpoint_consistency": 0.018642519941636237,
      "quadratic_transport_cost": 757.7133076985677
    },
```

```
    {
      "epoch": 49,
      "avg_loss": 0.09370880850996727
    },
    {
      "epoch": 50,
      "avg_loss": 0.09376188827694482,
      "paper_straightness": 0.0355519720663627,
      "endpoint_consistency": 0.017661075850924135,
      "quadratic_transport_cost": 760.2821756998698
    }
  ]

  Stage 2 history:
  [
    {
      "epoch": 1,
      "avg_loss": 0.009779529788324403
    },
    {
      "epoch": 2,
      "avg_loss": 0.007860557388108319,
      "paper_straightness": 0.006642800057306886,
      "endpoint_consistency": 0.0023987621418370964,
      "quadratic_transport_cost": 754.8248138427734
    },
    {
      "epoch": 3,
      "avg_loss": 0.007553118867123038
    },
    {
      "epoch": 4,
      "avg_loss": 0.007454652335240036,
      "paper_straightness": 0.003574325625474254,
      "endpoint_consistency": 0.001915495121920685,
      "quadratic_transport_cost": 751.0091400146484
    },
    {
      "epoch": 5,
      "avg_loss": 0.007039792359106115
    },
    {
      "epoch": 6,
      "avg_loss": 0.006769664634345497,
      "paper_straightness": 0.006848346131543319,
      "endpoint_consistency": 0.0019524491696287782,
      "quadratic_transport_cost": 752.2554931640625
    },
    {
      "epoch": 7,
      "avg_loss": 0.006815282554309974
    },
```

```
{
  "epoch": 8,
  "avg_loss": 0.006532328618236651,
  "paper_straightness": 0.006643092958256602,
  "endpoint_consistency": 0.001821714850947842,
  "quadratic_transport_cost": 751.2942657470703
},
{
  "epoch": 9,
  "avg_loss": 0.006498549559400377
},
{
  "epoch": 10,
  "avg_loss": 0.006259536442772732,
  "paper_straightness": 0.004487793776206672,
  "endpoint_consistency": 0.0016855459078541912,
  "quadratic_transport_cost": 753.5639801025391
},
{
  "epoch": 11,
  "avg_loss": 0.0064337540549249984
},
{
  "epoch": 12,
  "avg_loss": 0.0061261130354574155,
  "paper_straightness": 0.003349250454145173,
  "endpoint_consistency": 0.0016262887108879909,
  "quadratic_transport_cost": 751.1561584472656
},
{
  "epoch": 13,
  "avg_loss": 0.005988265834967646
},
{
  "epoch": 14,
  "avg_loss": 0.005934512228744789,
  "paper_straightness": 0.002758585847914219,
  "endpoint_consistency": 0.0015906797927200955,
  "quadratic_transport_cost": 747.3204956054688
},
{
  "epoch": 15,
  "avg_loss": 0.006335617075913036
},
{
  "epoch": 16,
  "avg_loss": 0.006050765221842971,
  "paper_straightness": 0.0026089221549530825,
  "endpoint_consistency": 0.0015737735392988839,
  "quadratic_transport_cost": 751.4380187988281
},
{
```

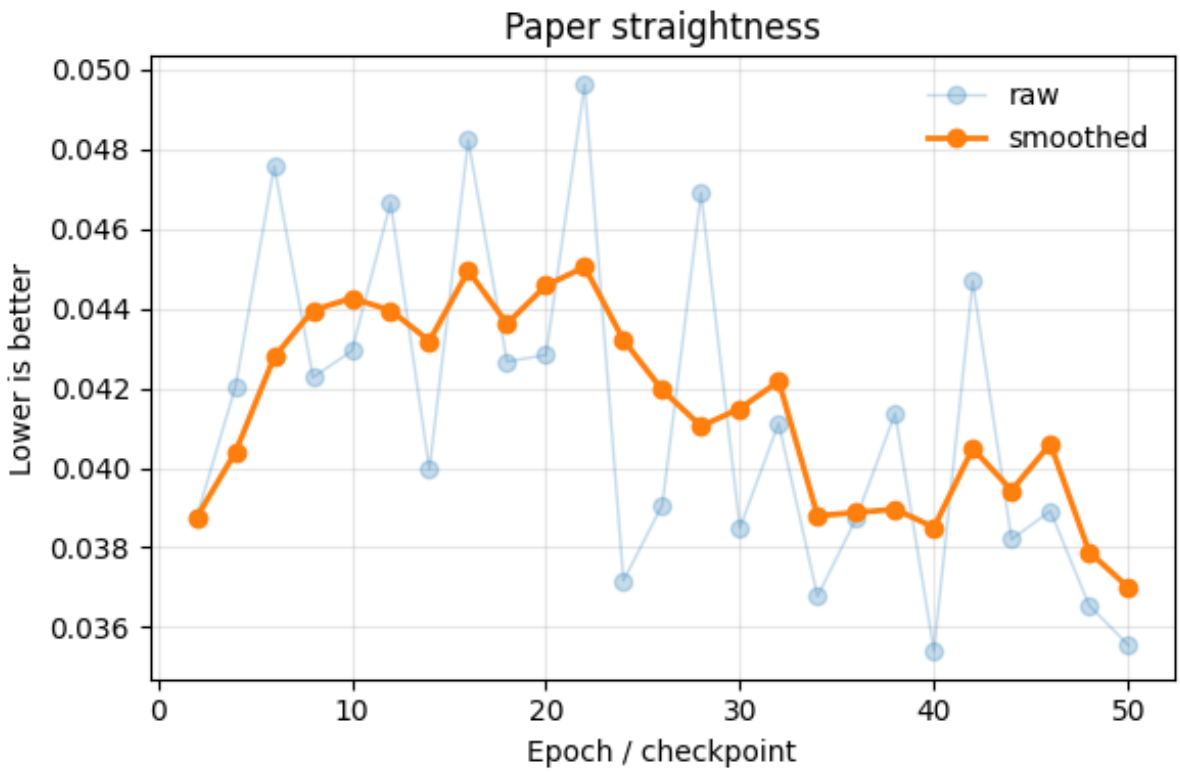
```
    "epoch": 17,
    "avg_loss": 0.006065569946375222
  },
  {
    "epoch": 18,
    "avg_loss": 0.005811217157906478,
    "paper_straightness": 0.005462131346575916,
    "endpoint_consistency": 0.0015344207709588886,
    "quadratic_transport_cost": 756.1489868164062
  },
  {
    "epoch": 19,
    "avg_loss": 0.005553104881889736
  },
  {
    "epoch": 20,
    "avg_loss": 0.005944782419778948,
    "paper_straightness": 0.0029220773528019586,
    "endpoint_consistency": 0.0014636376574550525,
    "quadratic_transport_cost": 752.8957672119141
  }
]
```

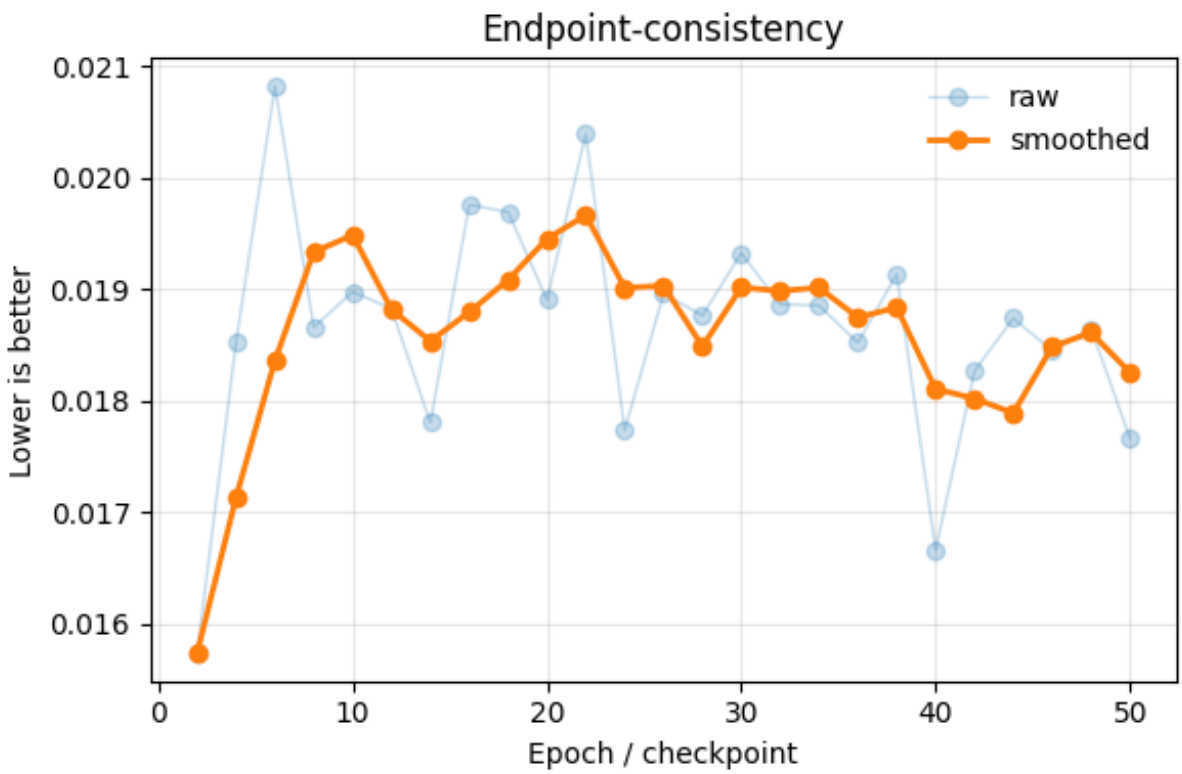

Block 15 — Plot Metrics vs Epoch / Checkpoint

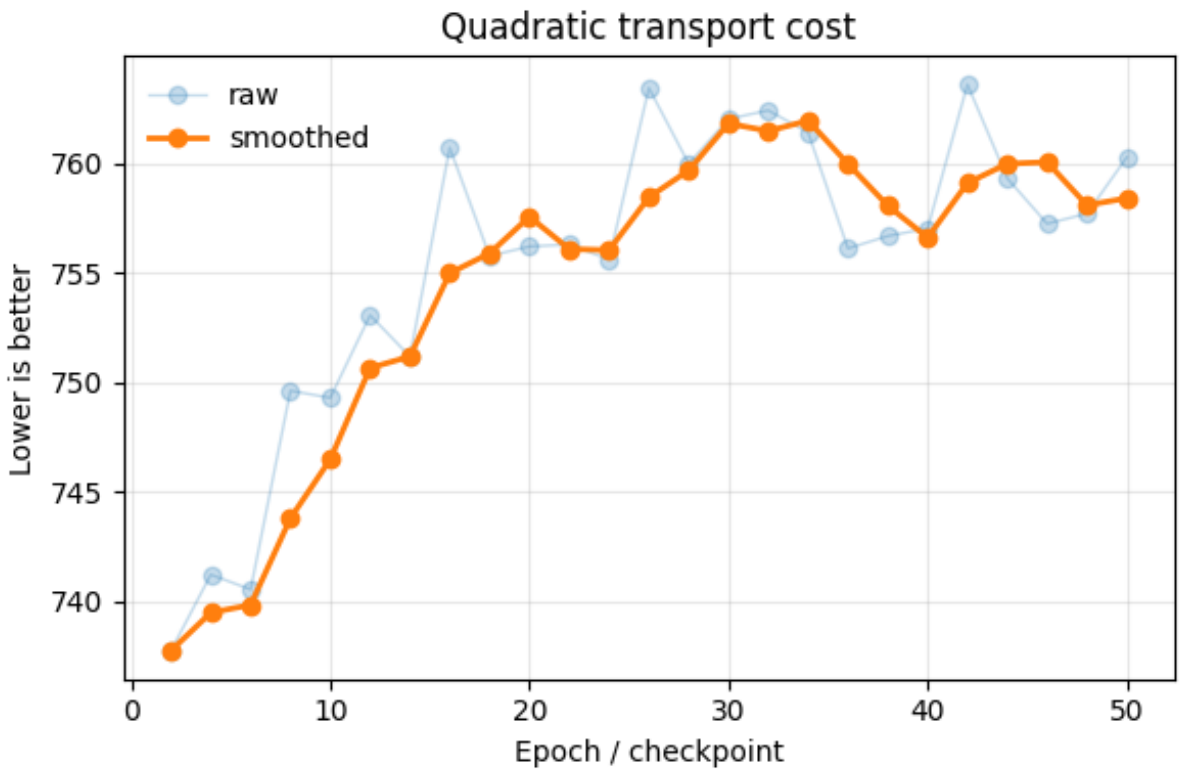
This cell turns the recorded histories into curves over training time. It overlays the raw tracked points with a light moving-average smoother so the plots are easier to read without changing the underlying saved metrics.

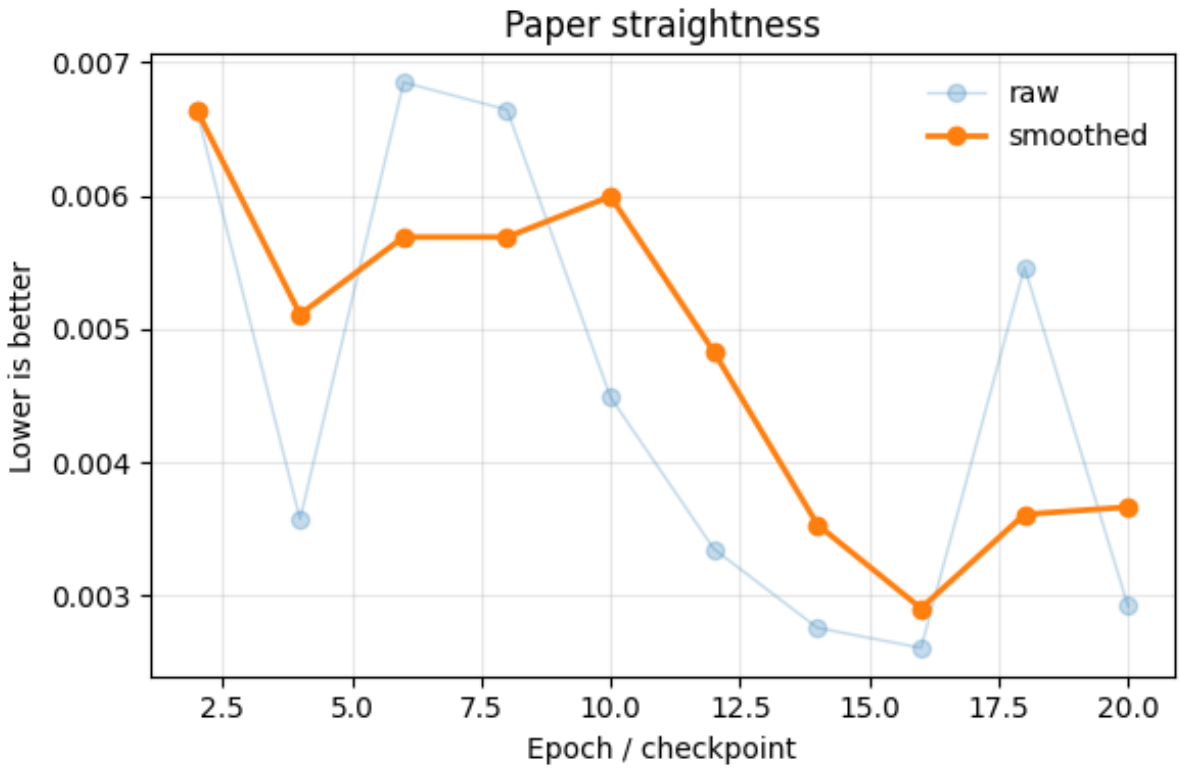
In [ ]:
```
# Block 15: Plot metric histories vs checkpoint/epoch

def smooth_history(values, window: int):
    if window <= 1 or len(values) <= 2:
        return np.asarray(values, dtype=float)
    smoothed = []
    for i in range(len(values)):
        lo = max(0, i - window + 1)
        smoothed.append(float(np.mean(values[lo : i + 1])))
    return np.asarray(smoothed, dtype=float)


def plot_metric_history(history, metric_key: str, stage_name: str, tit
    xs = [entry["epoch"] for entry in history if metric_key in entry]
    ys = [entry[metric_key] for entry in history if metric_key in entr
    if len(xs) == 0:
        print(f"No tracked values found for {stage_name} / {metric_key
        return

    raw = np.asarray(ys, dtype=float)
    smooth = smooth_history(raw, HISTORY_SMOOTHING_WINDOW)

    plt.figure(figsize=(6, 4))
```

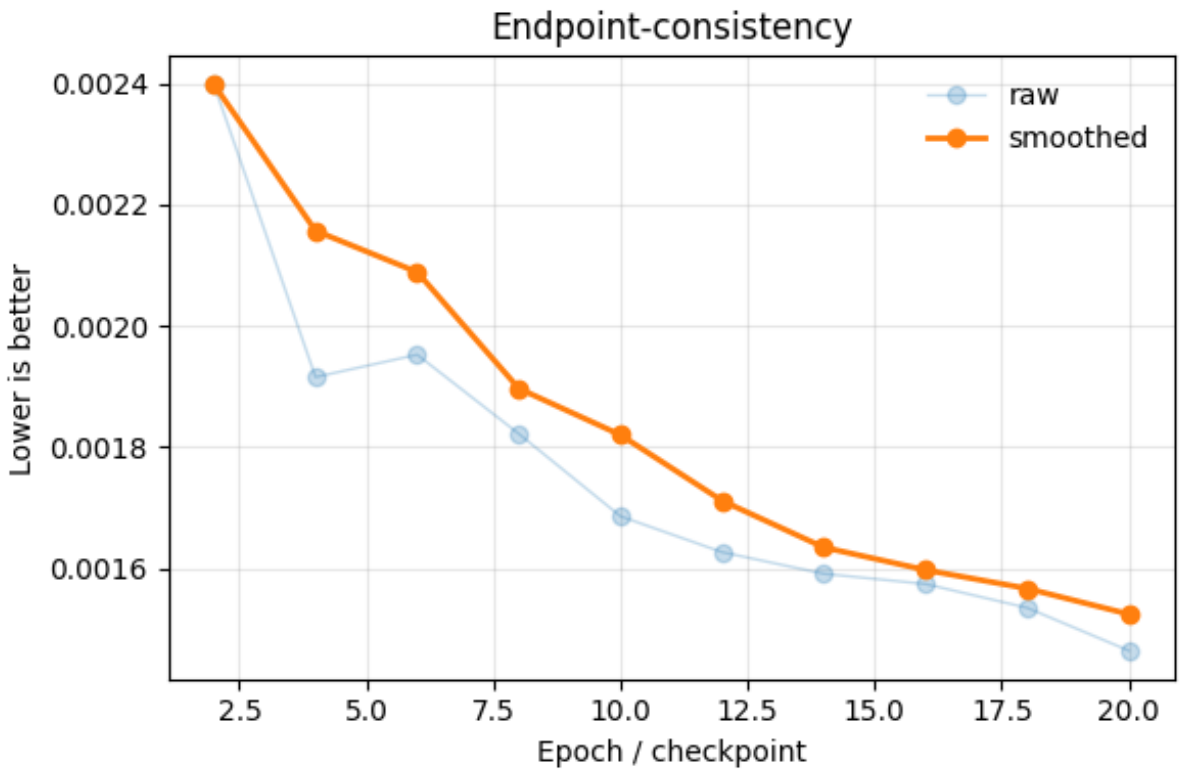
```
    plt.plot(xs, raw, marker="o", alpha=0.25, linewidth=1.0, label="ra
    plt.plot(xs, smooth, marker="o", linewidth=2.0, label="smoothed")
    plt.title(title)
    plt.xlabel("Epoch / checkpoint")
    plt.ylabel("Lower is better")
    plt.grid(True, alpha=0.3)
    plt.legend(frameon=False)
    plt.tight_layout()
    plt.show()

plot_metric_history(
    stage1_history,
    metric_key="paper_straightness",
    stage_name="stage1_cfm",
    title="Paper straightness",
)

plot_metric_history(
    stage1_history,
    metric_key="endpoint_consistency",
    stage_name="stage1_cfm",
    title="Endpoint-consistency",
)

plot_metric_history(
    stage1_history,
    metric_key="quadratic_transport_cost",
    stage_name="stage1_cfm",
    title="Quadratic transport cost",
)

plot_metric_history(
    stage2_history,
    metric_key="paper_straightness",
    stage_name="stage2_reflow",
    title="Paper straightness",
)

plot_metric_history(
    stage2_history,
    metric_key="endpoint_consistency",
    stage_name="stage2_reflow",
    title="Endpoint-consistency",
)

plot_metric_history(
    stage2_history,
    metric_key="quadratic_transport_cost",
    stage_name="stage2_reflow",
    title="Quadratic transport cost",
)
```

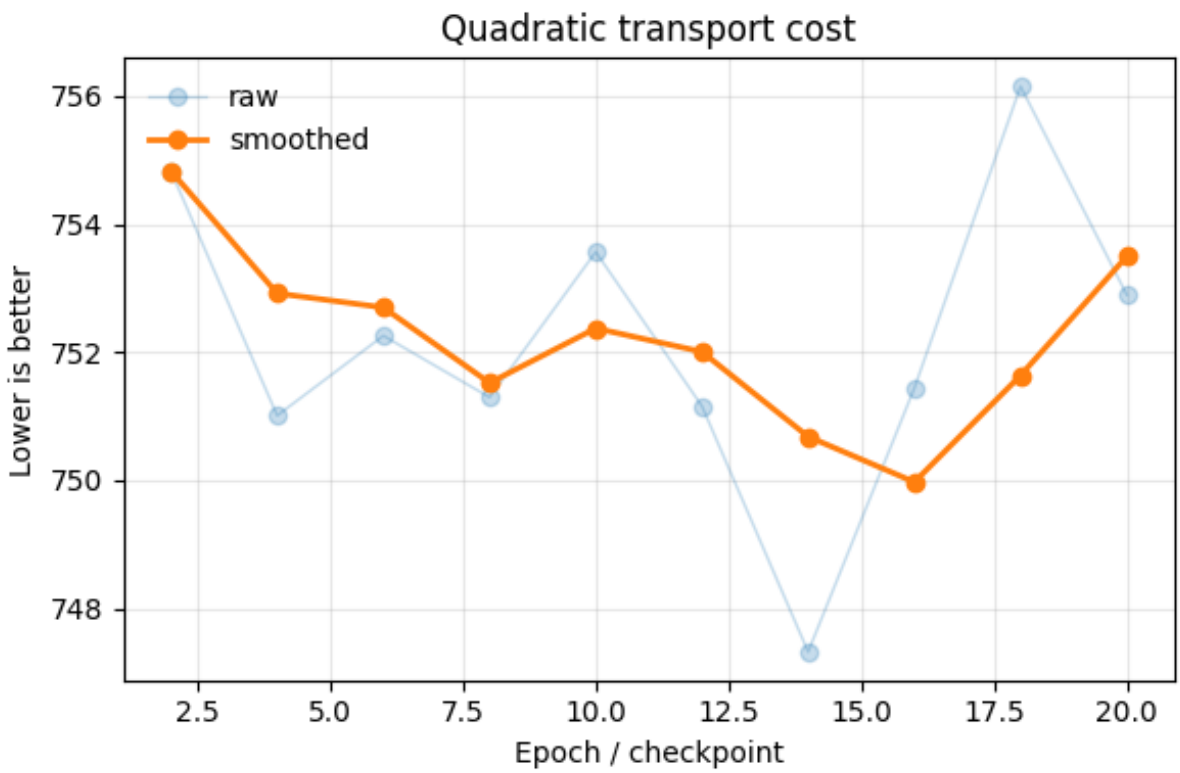

Paper straightness
raw
smoothed
Lower is better
0.050
0.048
0.046
0.044
0.042
0.040
0.038
0.036
0
10
20
30
40
50
Epoch / checkpoint


Endpoint-consistency
raw
smoothed
Lower is better
0.021
0.020
0.019
0.018
0.017
0.016
0
10
20
30
40
50
Epoch / checkpoint


Quadratic transport cost
raw
smoothed
Lower is better
760
755
750
745
740
0
10
20
30
40
50
Epoch / checkpoint


Paper straightness
raw
smoothed
Lower is better
0.007
0.006
0.005
0.004
0.003
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch / checkpoint





Block 15A — Save Checkpoint Metric Histories

This cell saves the tracked metric histories to disk so they can be reused later without rerunning the full notebook. This is helpful for reporting or for making external plots.

In [ ]:
```
# Block 15A: Save the checkpoint metric histories together

checkpoint_metric_histories = {
    "stage1_cfm": stage1_history,
    "stage2_reflow": stage2_history,
}

with open(OUT_DIR / "checkpoint_metric_histories.json", "w", encoding=
    json.dump(checkpoint_metric_histories, f, indent=2)

print("Saved:", OUT_DIR / "checkpoint_metric_histories.json")
```

```
Saved: mnist_rectified_flow_notebook_runs/checkpoint_metric_histories.j
son
```

Main One-Step Distillation after $k = 2$

This section adds the paper's **final one-step distillation stage** after the **2-Rectified Flow**.

The source flow here is the trained **stage-2 reflow model**.
We generate pairs [(Z_0^2, Z_1^2)] by sampling noise (Z_0^2) and integrating the stage-2 flow to obtain (Z_1^2). Then we train a direct predictor [\hat T_2(z_0)=z_0+v_ heta(z_0,0)] to match the endpoint in **one step**.

For this part:

- the **source flow's straightness** tells us whether one-step approximation should be easy,
- the distilled model is evaluated with **one-step endpoint MSE**.

Block 16 — Train One-Step Distillation after $k = 2$

We now distill the **2-Rectified Flow** into a one-step predictor. This is separate from reflow: it does **not** learn a new coupling or a new time-dependent flow. It simply learns to jump from the initial noise directly to the final endpoint of the already-rectified teacher flow.

In [ ]:
```
# Block 16: Train one-step distillation after k=2

stage2_distill_ckpt_dir = OUT_DIR / "stage2_distill_checkpoints"

stage2_distill_model, stage2_distill_losses, stage2_distill_history =
    source_flow=stage2_model,
    loader=train_loader,
    device=device,
    epochs=DISTILL_EPOCHS,
```

```
    lr=LR,
    stage_name="stage2_distill",
    pair_steps=PAIR_STEPS,
    pair_solver=PAIR_SOLVER,
    track_metrics=TRACK_METRICS,
    track_metric_every=TRACK_METRIC_EVERY,
    checkpoint_dir=stage2_distill_ckpt_dir if SAVE_EPOCH_CHECKPOINTS e
)

torch.save(
    {"model": stage2_distill_model.state_dict(), "losses": stage2_dist
    OUT_DIR / "stage2_distill.pt",
)
with open(OUT_DIR / "stage2_distill_history.json", "w", encoding="utf-
    json.dump(stage2_distill_history, f, indent=2)

print("Saved:", OUT_DIR / "stage2_distill.pt")
print("Saved:", OUT_DIR / "stage2_distill_history.json")
```

```
[stage2_distill] epoch 1/12 - loss: 0.006292
```

```
[stage2_distill] epoch 2/12 - loss: 0.005750 | one_step_endpoint_mse:
0.005517
```

```
[stage2_distill] epoch 3/12 - loss: 0.005557
```

```
[stage2_distill] epoch 4/12 - loss: 0.005406 | one_step_endpoint_mse:
0.005355
```

```
[stage2_distill] epoch 5/12 - loss: 0.005260
```

```
[stage2_distill] epoch 6/12 - loss: 0.005169 | one_step_endpoint_mse:
0.005111
```

```
[stage2_distill] epoch 7/12 - loss: 0.005066
```

```
[stage2_distill] epoch 8/12 - loss: 0.004988 | one_step_endpoint_mse:
0.004879
```

```
[stage2_distill] epoch 9/12 - loss: 0.004899
```

```
[stage2_distill] epoch 10/12 - loss: 0.004839 | one_step_endpoint_mse:
0.004777
```

```
[stage2_distill] epoch 11/12 - loss: 0.004776
```

```
[stage2_distill] epoch 12/12 - loss: 0.004720 | one_step_endpoint_mse:
0.004718
Saved: mnist_rectified_flow_notebook_runs/stage2_distill.pt
Saved: mnist_rectified_flow_notebook_runs/stage2_distill_history.json
```

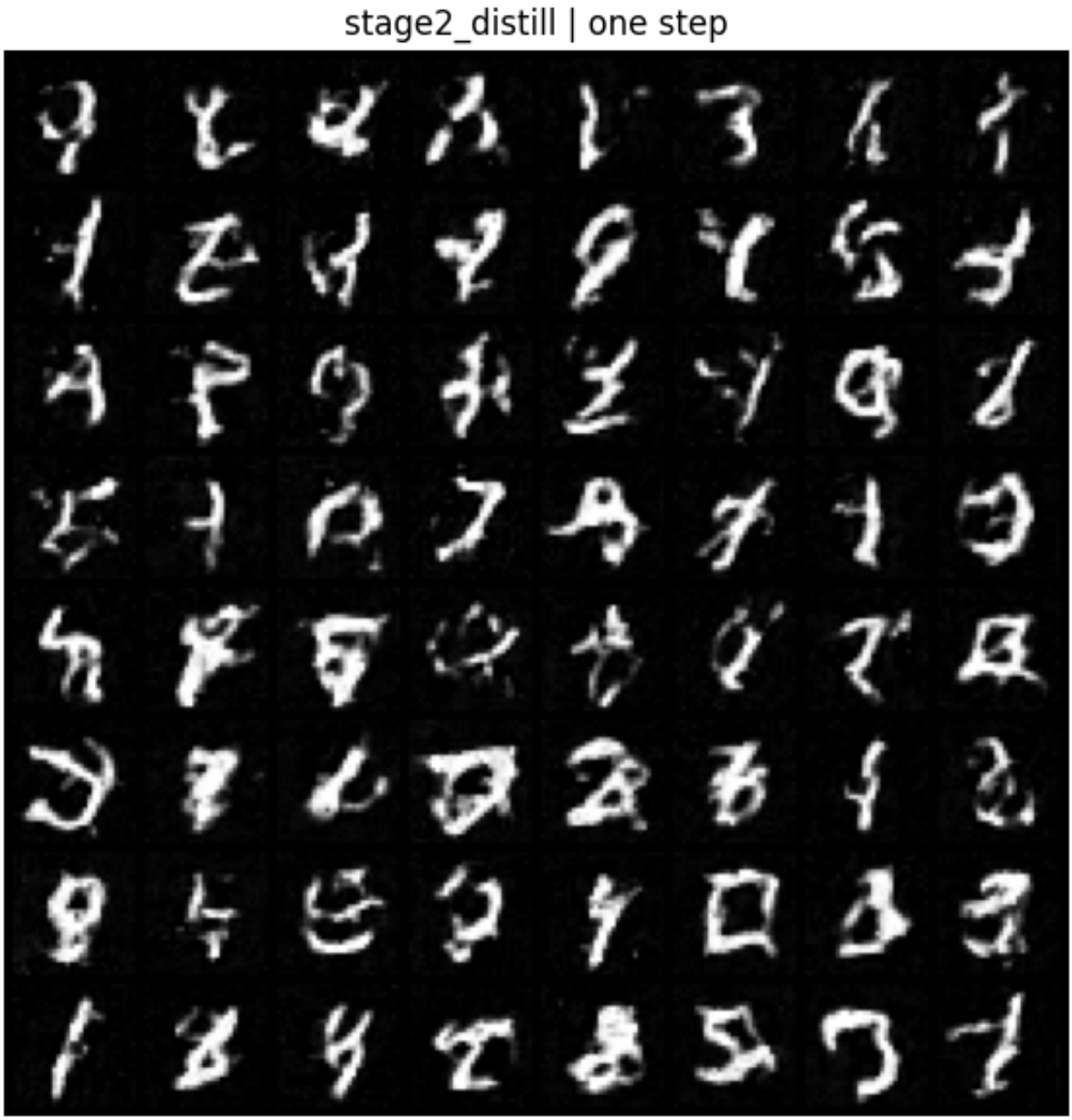
Block 16A — One-Step Samples after $k = 2$ Distillation

This block saves a **one-step sample grid** from the distilled model. At inference time, this model does not integrate the ODE. It directly applies the learned map [z_0 \mapsto \hat T_2(z_0).]

In [ ]:
```
# Block 16: One-step samples after k=2 distillation

stage2_distill_dir = OUT_DIR / "stage2_distill"
stage2_distill_dir.mkdir(parents=True, exist_ok=True)

stage2_distill_samples = sample_images_one_step(
    model=stage2_distill_model,
    n_samples=VIS_N,
    batch_size=VIS_N,
    device=device,
)
```

```
save_sample_grid(
    stage2_distill_samples,
    stage2_distill_dir / "samples_01_step.png",
    show_inline=SHOW_SAVED_SAMPLE_GRIDS,
    title="stage2_distill | one step",
)
print("Saved:", stage2_distill_dir / "samples_01_step.png")
```

```
Saved: mnist_rectified_flow_notebook_runs/stage2_distill/samples_01_step.png
```




```
Saved: mnist_rectified_flow_notebook_runs/stage2_distill/samples_01_step.png
```

Block 16C — Distillation Metrics after $k = 2$

For the source **2-Rectified Flow**, we keep reporting:

- the paper-style **straightness measure**,

- the earlier **endpoint-consistency** metric,
- the special **(t=0) endpoint-consistency**, which is exactly the one-step extrapolation error of the flow itself,
- and the **quadratic transport cost** (\mathbb{E}|Z_1-Z_0|^2), included as a special-case transport-cost diagnostic.

For the distilled one-step model, the main metric is: [\mathbb{E}\big[|Z_1^2-\hat T_2(Z_0^2)|^2\big],] which we call **one-step endpoint MSE**.

So rectification is still the part that improves **straightness**, while distillation uses that straightness to make **one-step inference** work better.

These values are now computed with larger Monte Carlo batches than the training-time checkpoints so the final comparison is more stable.

In [ ]:
```
# Block 16C: Distillation metrics after k=2

stage2_distill_results = {
    "stage2_reflow_source_flow": {
        "paper_straightness": estimate_paper_straightness(
            stage2_model,
            device=device,
            batch_size=EVAL_BATCH_SIZE,
            pair_steps=PAIR_STEPS,
            pair_solver=PAIR_SOLVER,
            batches=FINAL_METRIC_BATCHES,
        ),
        "endpoint_consistency": estimate_endpoint_consistency(
            stage2_model,
            device=device,
            batch_size=EVAL_BATCH_SIZE,
            num_steps=max(SAMPLE_STEPS),
            solver=SAMPLE_SOLVER,
            batches=max(2, FINAL_METRIC_BATCHES // 2),
        ),
        "endpoint_consistency_t0": estimate_t0_endpoint_consistency(
            stage2_model,
            device=device,
            batch_size=EVAL_BATCH_SIZE,
            pair_steps=PAIR_STEPS,
            pair_solver=PAIR_SOLVER,
            batches=FINAL_METRIC_BATCHES,
        ),
        "quadratic_transport_cost": estimate_quadratic_transport_cost(
            stage2_model,
            device=device,
            batch_size=EVAL_BATCH_SIZE,
            pair_steps=PAIR_STEPS,
            pair_solver=PAIR_SOLVER,
```

```
            batches=FINAL_METRIC_BATCHES,
        ),
    },
    "stage2_distilled_one_step": {
        "one_step_endpoint_mse": estimate_distilled_one_step_mse(
            distilled_model=stage2_distill_model,
            source_flow=stage2_model,
            device=device,
            batch_size=EVAL_BATCH_SIZE,
            pair_steps=PAIR_STEPS,
            pair_solver=PAIR_SOLVER,
            batches=FINAL_METRIC_BATCHES,
        ),
    },
}

print(json.dumps(stage2_distill_results, indent=2))

with open(OUT_DIR / "stage2_distill_metrics.json", "w", encoding="utf-
    json.dump(stage2_distill_results, f, indent=2)

print("Saved:", OUT_DIR / "stage2_distill_metrics.json")
```

```
{
  "stage2_reflow_source_flow": {
    "paper_straightness": 0.002623729268088937,
    "endpoint_consistency": 0.0014785800202503196,
    "endpoint_consistency_t0": 0.006939728232100606,
    "quadratic_transport_cost": 752.3766296386718
  },
  "stage2_distilled_one_step": {
    "one_step_endpoint_mse": 0.004729723837226629
  }
}
Saved: mnist_rectified_flow_notebook_runs/stage2_distill_metrics.json
```

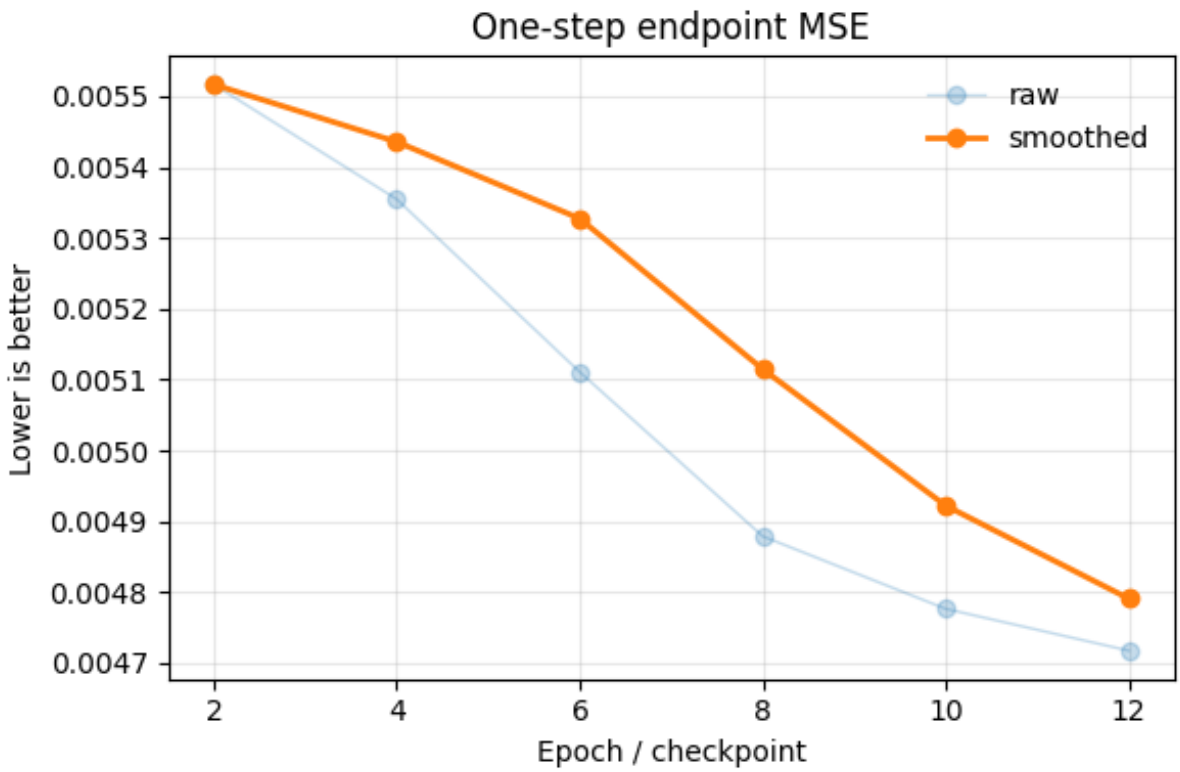
Block 16D — Distillation Plots and Summary for $k = 2$

This cell keeps the core distillation curve and prints the final summary dictionary. The source-flow transport cost remains available in the saved metrics dictionary, but it is no longer mixed into a shared bar chart with the smaller error metrics.

In [ ]:

```
# Block 16D: Distillation plots for k=2

plot_metric_history(
    stage2_distill_history,
    metric_key="one_step_endpoint_mse",
    stage_name="stage2_distill",
    title="One-step endpoint MSE",
)

print(json.dumps(stage2_distill_results, indent=2))
```

```
with open(OUT_DIR / "stage2_distill_metrics_pretty.json", "w", encodin
    json.dump(stage2_distill_results, f, indent=2)

print("Saved:", OUT_DIR / "stage2_distill_metrics_pretty.json")
```




```
{
  "stage2_reflow_source_flow": {
    "paper_straightness": 0.002623729268088937,
    "endpoint_consistency": 0.0014785800202503196,
    "endpoint_consistency_t0": 0.006939728232100606,
    "quadratic_transport_cost": 752.3766296386718
  },
  "stage2_distilled_one_step": {
    "one_step_endpoint_mse": 0.004729723837226629
  }
}
Saved: mnist_rectified_flow_notebook_runs/stage2_distill_metrics_prett
y.json
```

Optional Extension: Stage 3

Everything in this section is optional and comes **after** the priority experiments above. If enabled, we repeat the same concise set of plots for the 3-rectified model.

Block 16 — Optional Stage 3 Training ($k = 3$)

Everything from this point onward is optional. If enabled, this cell trains one more rectified model using stage 2 as the teacher, giving a simple demonstration of a third rectification step.

In [ ]:
```
# Block 16: Optional stage 3 training

if RUN_OPTIONAL_K3:
    stage3_ckpt_dir = OUT_DIR / "stage3_checkpoints"

    stage3_model, stage3_losses, stage3_history = train_reflow_stage(
        teacher=stage2_model,
        loader=train_loader,
        device=device,
        epochs=REFLOW_EPOCHS,
        lr=LR,
        stage_name="stage3_reflow",
        pair_steps=PAIR_STEPS,
        pair_solver=PAIR_SOLVER,
        track_metrics=TRACK_METRICS,
        track_metric_every=TRACK_METRIC_EVERY,
        track_straightness_batches=TRACK_STRAIGHTNESS_BATCHES,
        track_endpoint_batches=TRACK_ENDPOINT_BATCHES,
        track_endpoint_steps=TRACK_ENDPOINT_STEPS,
        checkpoint_dir=stage3_ckpt_dir if SAVE_EPOCH_CHECKPOINTS else
    )

    torch.save(
        {"model": stage3_model.state_dict(), "losses": stage3_losses,
        OUT_DIR / "stage3_reflow.pt",
    )
    with open(OUT_DIR / "stage3_history.json", "w", encoding="utf-8")
        json.dump(stage3_history, f, indent=2)

    print("Saved:", OUT_DIR / "stage3_reflow.pt")
    print("Saved:", OUT_DIR / "stage3_history.json")
else:
    stage3_model, stage3_losses, stage3_history = None, [], []
    print("RUN_OPTIONAL_K3 is False, so stage 3 is skipped.")
```

```
[stage3_reflow] epoch 1/20 - loss: 0.003452
```

```
[stage3_reflow] epoch 2/20 - loss: 0.003450 | straightness: 0.004178 |
endpoint_consistency: 0.000982 | quadratic_transport_cost: 747.544891
```

```
[stage3_reflow] epoch 3/20 - loss: 0.003347
```

```
[stage3_reflow] epoch 4/20 - loss: 0.003270 | straightness: 0.002469 |
endpoint_consistency: 0.000871 | quadratic_transport_cost: 747.820541
```

```
[stage3_reflow] epoch 5/20 - loss: 0.003225
```

```
[stage3_reflow] epoch 6/20 - loss: 0.003089 | straightness: 0.001471 |
endpoint_consistency: 0.000804 | quadratic_transport_cost: 748.180710
```

```
[stage3_reflow] epoch 7/20 - loss: 0.002753
```

```
[stage3_reflow] epoch 8/20 - loss: 0.003193 | straightness: 0.001786 |
endpoint_consistency: 0.000818 | quadratic_transport_cost: 749.921371
```

```
[stage3_reflow] epoch 9/20 - loss: 0.003212
```

```
[stage3_reflow] epoch 10/20 - loss: 0.003071 | straightness: 0.002027 |
endpoint_consistency: 0.000826 | quadratic_transport_cost: 747.301773
```

```
[stage3_reflow] epoch 11/20 - loss: 0.003152
```

```
[stage3_reflow] epoch 12/20 - loss: 0.003152 | straightness: 0.005146 |
endpoint_consistency: 0.000745 | quadratic_transport_cost: 748.213928
```

```
[stage3_reflow] epoch 13/20 - loss: 0.003111
```

```
[stage3_reflow] epoch 14/20 - loss: 0.002892 | straightness: 0.002101 |
endpoint_consistency: 0.000764 | quadratic_transport_cost: 749.644897
```

```
[stage3_reflow] epoch 15/20 - loss: 0.002825
```

```
[stage3_reflow] epoch 16/20 - loss: 0.003241 | straightness: 0.001591 |
endpoint_consistency: 0.000950 | quadratic_transport_cost: 747.121078
```

```
[stage3_reflow] epoch 17/20 - loss: 0.003123
```

```
[stage3_reflow] epoch 18/20 - loss: 0.003209 | straightness: 0.004260 |
endpoint_consistency: 0.000745 | quadratic_transport_cost: 747.864334
```

```
[stage3_reflow] epoch 19/20 - loss: 0.003100
```

```
[stage3_reflow] epoch 20/20 - loss: 0.003000 | straightness: 0.002887 |
endpoint_consistency: 0.000800 | quadratic_transport_cost: 743.502548
Saved: mnist_rectified_flow_notebook_runs/stage3_reflow.pt
Saved: mnist_rectified_flow_notebook_runs/stage3_history.json
```

Block 18 — Optional Qualitative Comparison Including Stage 3

If stage 3 is enabled, this cell extends the NFE sample-grid comparison to include the 3-rectified model. This is useful for checking whether an additional rectification step gives any visible benefit on MNIST.

In [ ]:
```
# Block 18: Optional qualitative comparison including stage 3

if RUN_OPTIONAL_K3 and stage3_model is not None:
    stage3_models = {
        "stage1_cfm": stage1_model.eval(),
        "stage2_reflow": stage2_model.eval(),
        "stage3_reflow": stage3_model.eval(),
    }

    for stage_name, model in stage3_models.items():
        stage_dir = OUT_DIR / stage_name
        stage_dir.mkdir(parents=True, exist_ok=True)
        for steps in SAMPLE_STEPS:
            samples = sample_images(
                model=model,
                n_samples=VIS_N,
                batch_size=VIS_N,
                num_steps=steps,
                device=device,
                solver=SAMPLE_SOLVER,
            )
            save_sample_grid(
                samples,
                stage_dir / f"samples_{steps:02d}_steps_k3_run.png",
                show_inline=SHOW_SAVED_SAMPLE_GRIDS and (steps in INLI
                title=f"{stage_name} | NFE={steps}",
            )
```

Output hidden; open in https://colab.research.google.com to view.

Block 19 — Optional Quantitative Comparison Including Stage 3

This cell computes the same three flow-side metrics for the stage-3 model and compares them against stages 1 and 2. It mirrors the earlier quantitative block but now includes the optional extension.

In [ ]:
```
# Block 19: Optional quantitative comparison including stage 3

if RUN_OPTIONAL_K3 and stage3_model is not None:
    k3_results = {}
    stage3_models = {
        "stage1_cfm": stage1_model.eval(),
        "stage2_reflow": stage2_model.eval(),
        "stage3_reflow": stage3_model.eval(),
    }

    for stage_name, model in stage3_models.items():
        k3_results[stage_name] = {
            "paper_straightness": estimate_paper_straightness(
                model,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
```

```
                pair_steps=PAIR_STEPS,
                pair_solver=PAIR_SOLVER,
                batches=FINAL_METRIC_BATCHES,
            ),
            "endpoint_consistency": estimate_endpoint_consistency(
                model,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
                num_steps=max(SAMPLE_STEPS),
                solver=SAMPLE_SOLVER,
                batches=max(2, FINAL_METRIC_BATCHES // 2),
            ),
            "quadratic_transport_cost": estimate_quadratic_transport_c
                model,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
                pair_steps=PAIR_STEPS,
                pair_solver=PAIR_SOLVER,
                batches=FINAL_METRIC_BATCHES,
            ),
        }

    print(json.dumps(k3_results, indent=2))

    with open(OUT_DIR / "optional_k3_metrics.json", "w", encoding="utf
        json.dump(k3_results, f, indent=2)

    print("Saved:", OUT_DIR / "optional_k3_metrics.json")
```

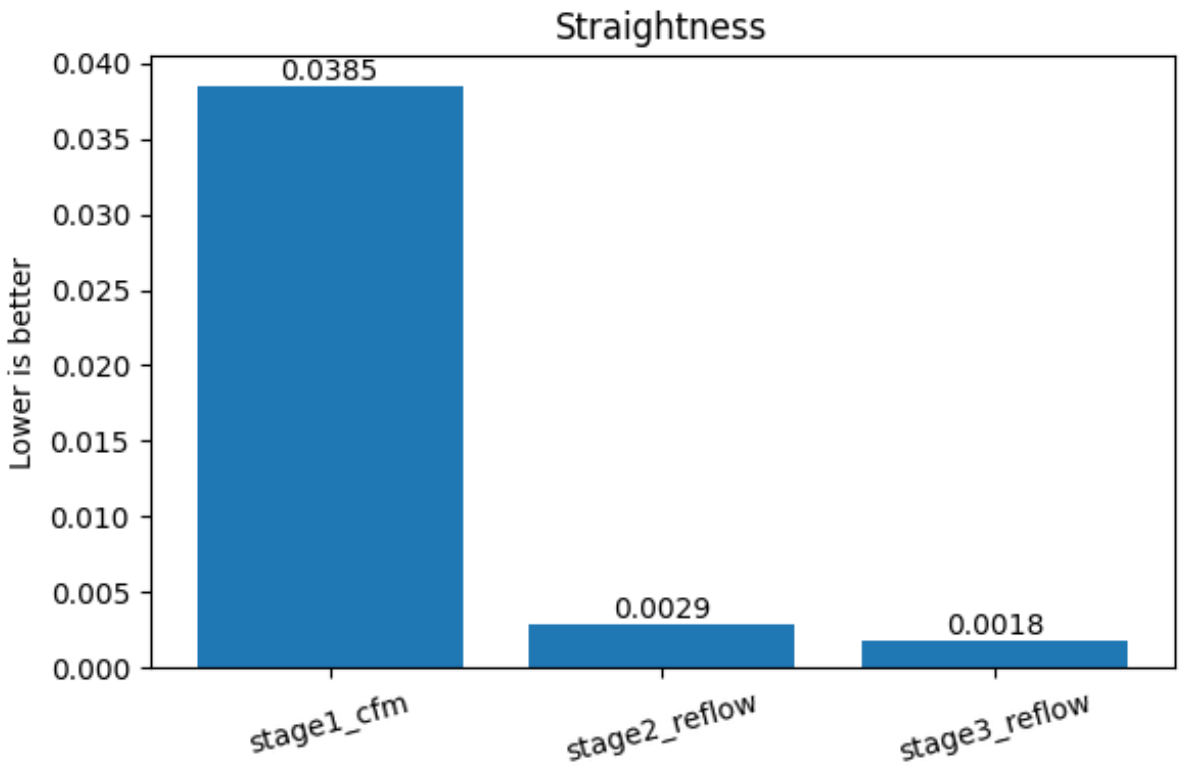
```
{
  "stage1_cfm": {
    "paper_straightness": 0.038504932820796964,
    "endpoint_consistency": 0.017593241873419173,
    "quadratic_transport_cost": 760.5081481933594
  },
  "stage2_reflow": {
    "paper_straightness": 0.0028920436976477503,
    "endpoint_consistency": 0.0014853227907252631,
    "quadratic_transport_cost": 752.2377502441407
  },
  "stage3_reflow": {
    "paper_straightness": 0.001779099158011377,
    "endpoint_consistency": 0.000800001332214606,
    "quadratic_transport_cost": 744.4775939941406
  }
}
Saved: mnist_rectified_flow_notebook_runs/optional_k3_metrics.json
```

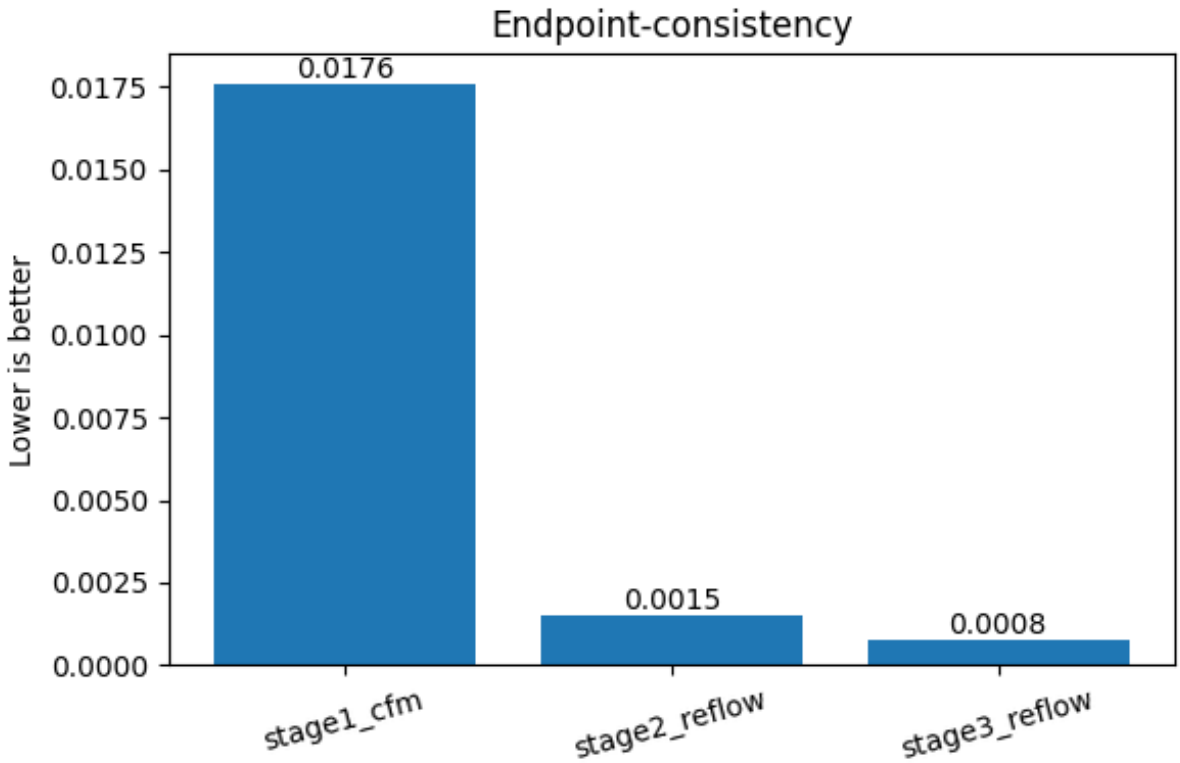
Block 19A — Optional Metric Plots Including Stage 3

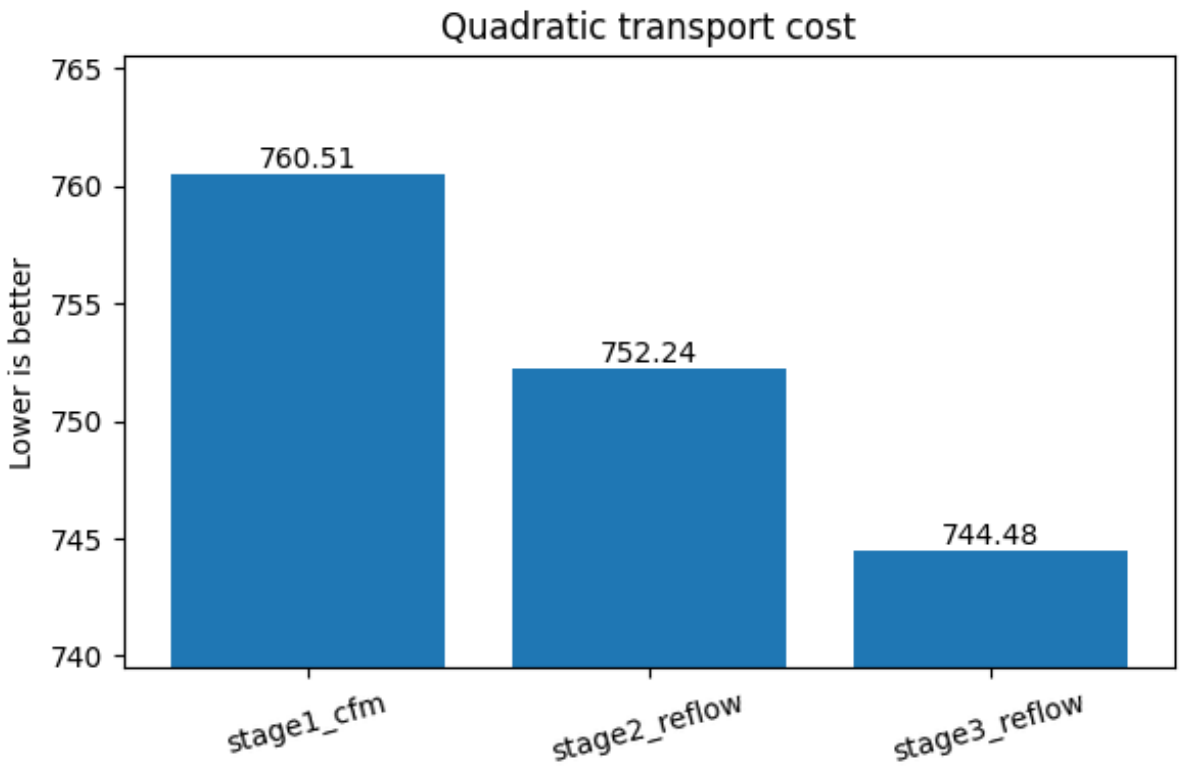
If stage 3 is enabled, this cell plots the final values of the same three flow-side metrics across all three stages. This helps show whether the optional extra

rectification step continues the same trend.

In [ ]:
```
# Block 19A: Optional compact metric plots including stage 3

if RUN_OPTIONAL_K3 and stage3_model is not None:
    stage_names = list(k3_results.keys())
    straightness_vals = [k3_results[s]["paper_straightness"] for s in
    endpoint_vals = [k3_results[s]["endpoint_consistency"] for s in st
    transport_vals = [k3_results[s]["quadratic_transport_cost"] for s

    # Paper straightness
    plt.figure(figsize=(6, 4))
    bars = plt.bar(stage_names, straightness_vals)
    plt.title("Straightness")
    plt.ylabel("Lower is better")
    plt.xticks(rotation=15)
    for b, v in zip(bars, straightness_vals):
        plt.text(b.get_x() + b.get_width() / 2, v, f"{v:.4f}", ha="cen
    plt.tight_layout()
    plt.show()

    # Endpoint-consistency
    plt.figure(figsize=(6, 4))
    bars = plt.bar(stage_names, endpoint_vals)
    plt.title("Endpoint-consistency")
    plt.ylabel("Lower is better")
    plt.xticks(rotation=15)
    for b, v in zip(bars, endpoint_vals):
        plt.text(b.get_x() + b.get_width() / 2, v, f"{v:.4f}", ha="cen
    plt.tight_layout()
    plt.show()

    # Quadratic transport cost
    plt.figure(figsize=(6, 4))
    bars = plt.bar(stage_names, transport_vals)
    plt.title("Quadratic transport cost")
    plt.ylabel("Lower is better")
    plt.xticks(rotation=15)

    ymin = min(transport_vals) - 5
    ymax = max(transport_vals) + 5
    plt.ylim(ymin, ymax)

    for b, v in zip(bars, transport_vals):
        plt.text(b.get_x() + b.get_width() / 2, v, f"{v:.2f}", ha="cen

    plt.tight_layout()
    plt.show()
else:
    print("Skipping optional stage-3 metric plots because RUN_OPTIONAL
```


Straightness
Lower is better
0.040
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000
0.0385
0.0029
0.0018
stage1_cfm
stage2_reflow
stage3_reflow


Endpoint-consistency
Lower is better
0.0175
0.0150
0.0125
0.0100
0.0075
0.0050
0.0025
0.0000
0.0176
0.0015
0.0008
stage1_cfm
stage2_reflow
stage3_reflow

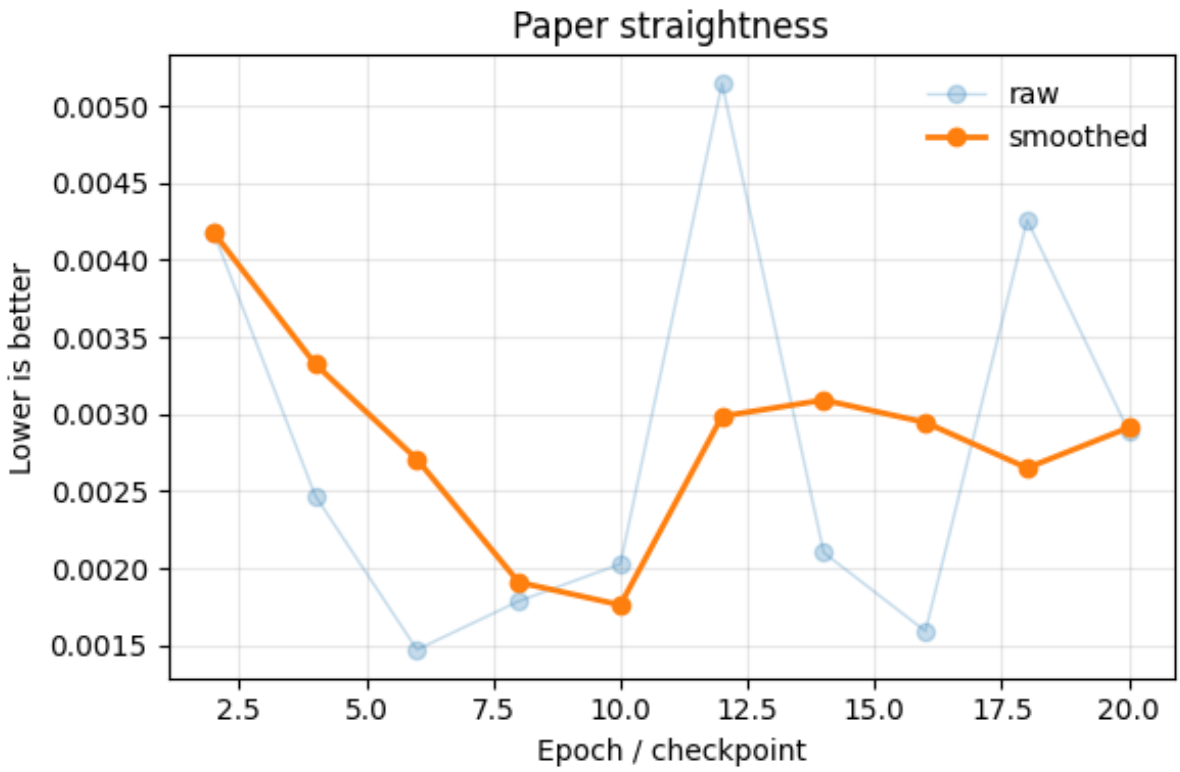


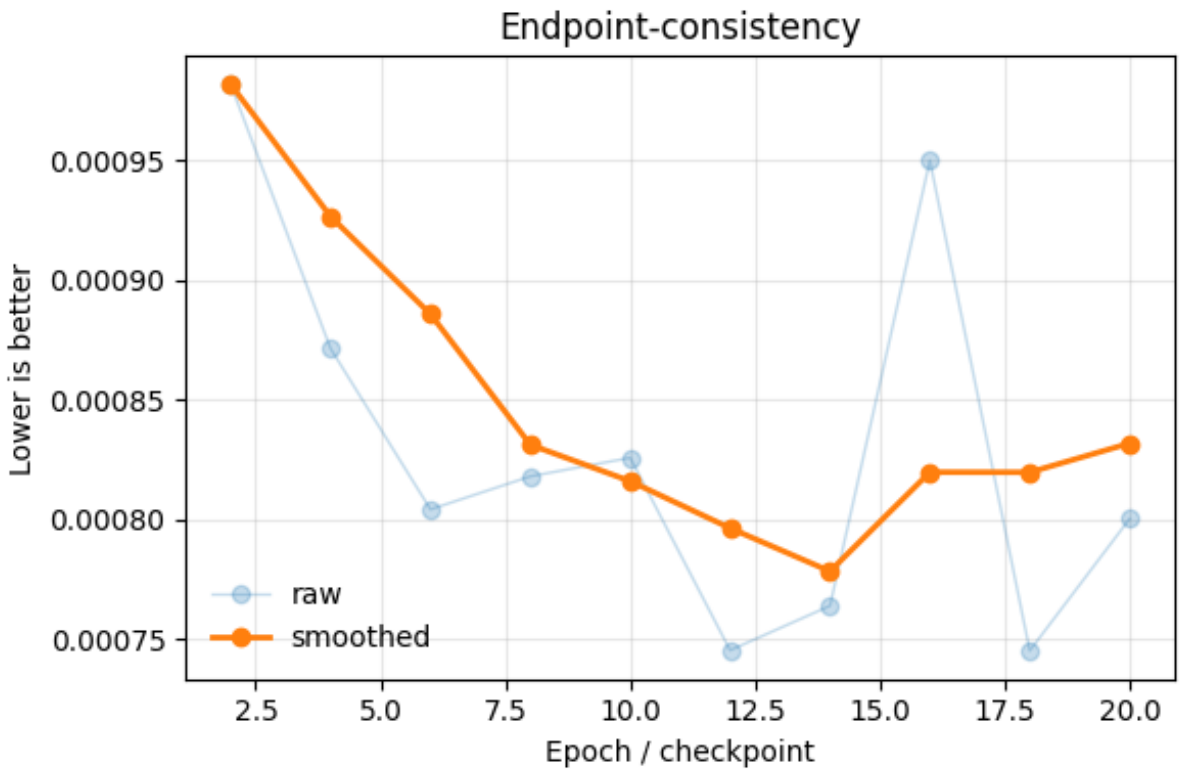
Block 19B — Optional Stage-3 Metric Curves

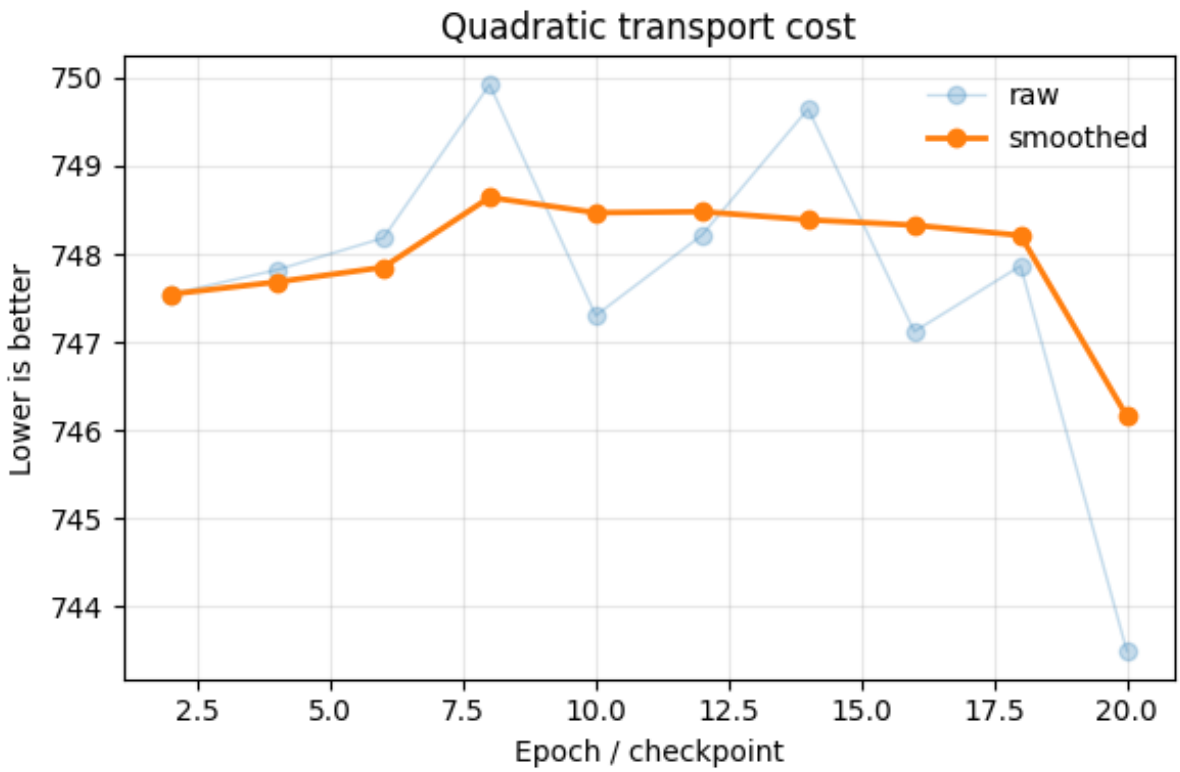
This cell plots the checkpoint-wise straightness and endpoint-consistency curves for stage 3. It is the optional analogue of the main training-progress plots above.

In [ ]:
```
# Block 19B: Optional checkpoint-metric plots for stage 3

if RUN_OPTIONAL_K3 and stage3_model is not None:
    plot_metric_history(
        stage3_history,
        metric_key="paper_straightness",
        stage_name="stage3_reflow",
        title="Paper straightness",
    )

    plot_metric_history(
        stage3_history,
        metric_key="endpoint_consistency",
        stage_name="stage3_reflow",
        title="Endpoint-consistency",
    )

    plot_metric_history(
        stage3_history,
        metric_key="quadratic_transport_cost",
        stage_name="stage3_reflow",
        title="Quadratic transport cost",
    )
else:
```

```
    print("Skipping optional stage-3 checkpoint curves because RUN_OPT
```







Optional One-Step Distillation after $k = 3$

If you run the optional **3-Rectified Flow**, you can also apply the same final one-step distillation idea there.

The logic is the same as for (k=2):

- first obtain a straighter source flow by rectification,
- then distill its endpoint coupling into a direct one-step predictor.

Block 22A — Optional Train One-Step Distillation after $k = 3$

This block is only active when the optional stage-3 reflow is enabled.

In [ ]:
```
# Block 22A: Optional train one-step distillation after k=3

if RUN_OPTIONAL_K3 and stage3_model is not None:
    stage3_distill_ckpt_dir = OUT_DIR / "stage3_distill_checkpoints"

    stage3_distill_model, stage3_distill_losses, stage3_distill_histor
        source_flow=stage3_model,
        loader=train_loader,
        device=device,
        epochs=DISTILL_EPOCHS,
        lr=LR,
        stage_name="stage3_distill",
```

```
        pair_steps=PAIR_STEPS,
        pair_solver=PAIR_SOLVER,
        track_metrics=TRACK_METRICS,
        track_metric_every=TRACK_METRIC_EVERY,
        checkpoint_dir=stage3_distill_ckpt_dir if SAVE_EPOCH_CHECKPOIN
    )

    torch.save(
        {"model": stage3_distill_model.state_dict(), "losses": stage3_
        OUT_DIR / "stage3_distill.pt",
    )
    with open(OUT_DIR / "stage3_distill_history.json", "w", encoding="
        json.dump(stage3_distill_history, f, indent=2)

    print("Saved:", OUT_DIR / "stage3_distill.pt")
    print("Saved:", OUT_DIR / "stage3_distill_history.json")
else:
    stage3_distill_model, stage3_distill_losses, stage3_distill_histor
    print("RUN_OPTIONAL_K3 is False, so optional k=3 distillation is s
```

```
[stage3_distill] epoch 1/12 - loss: 0.003724
```

```
[stage3_distill] epoch 2/12 - loss: 0.003391 | one_step_endpoint_mse:
0.003237
```

```
[stage3_distill] epoch 3/12 - loss: 0.003283
```

```
[stage3_distill] epoch 4/12 - loss: 0.003192 | one_step_endpoint_mse:
0.003293
```

```
[stage3_distill] epoch 5/12 - loss: 0.003113
```

```
[stage3_distill] epoch 6/12 - loss: 0.003055 | one_step_endpoint_mse:
0.003043
```

```
[stage3_distill] epoch 7/12 - loss: 0.003000
```

```
[stage3_distill] epoch 8/12 - loss: 0.002945 | one_step_endpoint_mse:
0.002874
```

```
[stage3_distill] epoch 9/12 - loss: 0.002908
```

```
[stage3_distill] epoch 10/12 - loss: 0.002872 | one_step_endpoint_mse:
0.002852
```

```
[stage3_distill] epoch 11/12 - loss: 0.002830
```

```
[stage3_distill] epoch 12/12 - loss: 0.002793 | one_step_endpoint_mse:
0.002682
Saved: mnist_rectified_flow_notebook_runs/stage3_distill.pt
Saved: mnist_rectified_flow_notebook_runs/stage3_distill_history.json
```

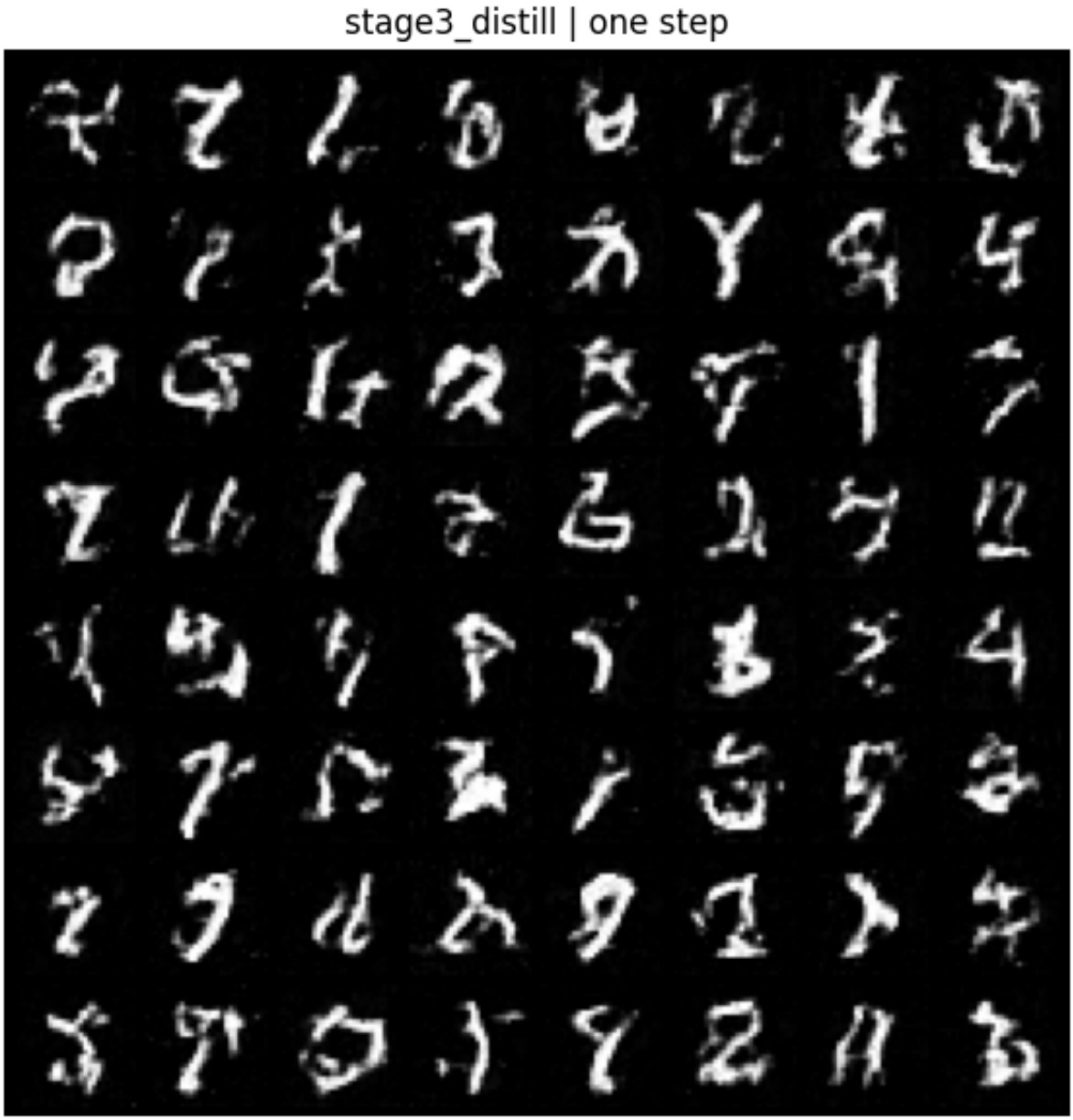
Block 22B — Optional One-Step Samples after $k = 3$ Distillation

If stage 3 is enabled, this saves a one-step sample grid from the distilled (k=3) predictor.

In [ ]:
```
# Block 22B: Optional one-step samples after k=3 distillation

if RUN_OPTIONAL_K3 and stage3_distill_model is not None:
    stage3_distill_dir = OUT_DIR / "stage3_distill"
    stage3_distill_dir.mkdir(parents=True, exist_ok=True)

    stage3_distill_samples = sample_images_one_step(
        model=stage3_distill_model,
        n_samples=VIS_N,
        batch_size=VIS_N,
        device=device,
    )
```

```
    save_sample_grid(
        stage3_distill_samples,
        stage3_distill_dir / "samples_01_step.png",
        show_inline=SHOW_SAVED_SAMPLE_GRIDS,
        title="stage3_distill | one step",
    )
    print("Saved:", stage3_distill_dir / "samples_01_step.png")
else:
    print("Skipping optional k=3 one-step sample grid.")
```

```
Saved: mnist_rectified_flow_notebook_runs/stage3_distill/samples_01_step.png
```




```
Saved: mnist_rectified_flow_notebook_runs/stage3_distill/samples_01_step.png
```

Block 22C — Optional Distillation Metrics after $k = 3$

As before, the source flow keeps the **straightness** interpretation and also carries

the **quadratic transport cost** diagnostic, while the distilled model is evaluated by **one-step endpoint MSE**.

As above, the final numbers here use larger Monte Carlo batches than the checkpoint curves.

In [ ]:
```
# Block 22C: Optional distillation metrics after k=3

if RUN_OPTIONAL_K3 and stage3_model is not None and stage3_distill_mod
    stage3_distill_results = {
        "stage3_reflow_source_flow": {
            "paper_straightness": estimate_paper_straightness(
                stage3_model,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
                pair_steps=PAIR_STEPS,
                pair_solver=PAIR_SOLVER,
                batches=FINAL_METRIC_BATCHES,
            ),
            "endpoint_consistency": estimate_endpoint_consistency(
                stage3_model,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
                num_steps=max(SAMPLE_STEPS),
                solver=SAMPLE_SOLVER,
                batches=max(2, FINAL_METRIC_BATCHES // 2),
            ),
            "endpoint_consistency_t0": estimate_t0_endpoint_consistenc
                stage3_model,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
                pair_steps=PAIR_STEPS,
                pair_solver=PAIR_SOLVER,
                batches=FINAL_METRIC_BATCHES,
            ),
            "quadratic_transport_cost": estimate_quadratic_transport_c
                stage3_model,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
                pair_steps=PAIR_STEPS,
                pair_solver=PAIR_SOLVER,
                batches=FINAL_METRIC_BATCHES,
            ),
        },
        "stage3_distilled_one_step": {
            "one_step_endpoint_mse": estimate_distilled_one_step_mse(
                distilled_model=stage3_distill_model,
                source_flow=stage3_model,
                device=device,
                batch_size=EVAL_BATCH_SIZE,
                pair_steps=PAIR_STEPS,
```

```
                pair_solver=PAIR_SOLVER,
                batches=FINAL_METRIC_BATCHES,
            ),
        },
    }

    print(json.dumps(stage3_distill_results, indent=2))

    with open(OUT_DIR / "stage3_distill_metrics.json", "w", encoding="
        json.dump(stage3_distill_results, f, indent=2)

    print("Saved:", OUT_DIR / "stage3_distill_metrics.json")
else:
    stage3_distill_results = {}
    print("Skipping optional k=3 distillation metrics.")
```

```
{
  "stage3_reflow_source_flow": {
    "paper_straightness": 0.0021360276150517166,
    "endpoint_consistency": 0.0007978690752139528,
    "endpoint_consistency_t0": 0.004253851436078549,
    "quadratic_transport_cost": 745.0741577148438
  },
  "stage3_distilled_one_step": {
    "one_step_endpoint_mse": 0.0027493155328556897
  }
}
Saved: mnist_rectified_flow_notebook_runs/stage3_distill_metrics.json
```

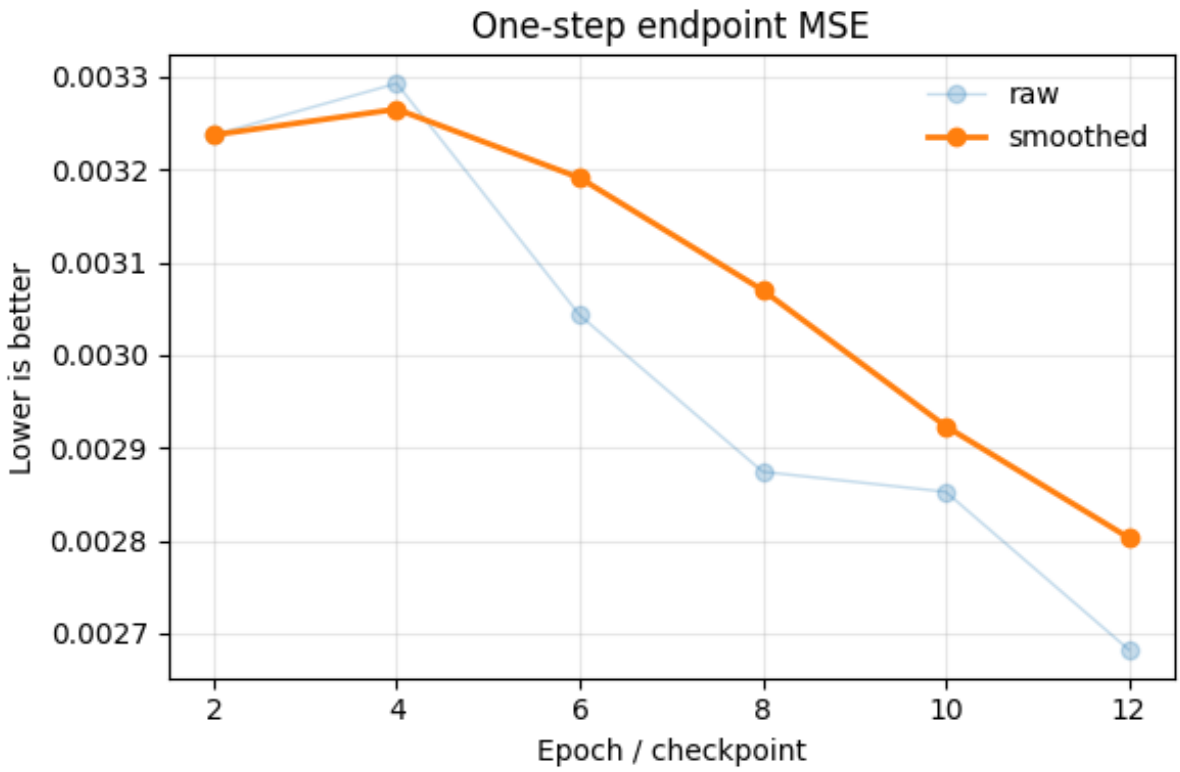
Block 22D — Optional Distillation Plots and Summary for $k = 3$

This cell keeps the core one-step endpoint-MSE curve for the optional $k = 3$ distillation stage and prints the final summary dictionary instead of mixing different scales in one bar chart.

In [ ]:

```
# Block 22D: Optional distillation plots for k=3

if RUN_OPTIONAL_K3 and stage3_distill_model is not None:
    plot_metric_history(
        stage3_distill_history,
        metric_key="one_step_endpoint_mse",
        stage_name="stage3_distill",
        title="One-step endpoint MSE",
    )

    print(json.dumps(stage3_distill_results, indent=2))

    with open(OUT_DIR / "stage3_distill_metrics_pretty.json", "w", enc
        json.dump(stage3_distill_results, f, indent=2)

    print("Saved:", OUT_DIR / "stage3_distill_metrics_pretty.json")
else:
```

```
    print("Skipping optional k=3 distillation plots.")
```




```
{
  "stage3_reflow_source_flow": {
    "paper_straightness": 0.0021360276150517166,
    "endpoint_consistency": 0.0007978690752139528,
    "endpoint_consistency_t0": 0.004253851436078549,
    "quadratic_transport_cost": 745.0741577148438
  },
  "stage3_distilled_one_step": {
    "one_step_endpoint_mse": 0.0027493155328556897
  }
}
Saved: mnist_rectified_flow_notebook_runs/stage3_distill_metrics_prett
y.json
```

Block 23 — Final Save Bundle

This final cell packages the main results, histories, and configuration into files saved under the output directory. It gives you one compact place to look for all final artifacts from the run.

In [ ]:
```
# Block 23: save

summary_bundle = {
    "stage1_losses": stage1_losses,
    "stage2_losses": stage2_losses,
    "stage1_history": stage1_history,
    "stage2_history": stage2_history,
    "main_results": main_results,
```

```
    "stage2_distill_losses": stage2_distill_losses,
    "stage2_distill_history": stage2_distill_history,
    "stage2_distill_results": stage2_distill_results,
    "config": {
        "seed": SEED,
        "batch_size": BATCH_SIZE,
        "hidden_dim": HIDDEN_DIM,
        "n_layers": N_LAYERS,
        "lr": LR,
        "cfm_epochs": CFM_EPOCHS,
        "reflow_epochs": REFLOW_EPOCHS,
        "distill_epochs": DISTILL_EPOCHS,
        "pair_steps": PAIR_STEPS,
        "pair_solver": PAIR_SOLVER,
        "sample_solver": SAMPLE_SOLVER,
        "sample_steps": SAMPLE_STEPS,
        "track_metrics": TRACK_METRICS,
        "track_metric_every": TRACK_METRIC_EVERY,
        "track_straightness_batches": TRACK_STRAIGHTNESS_BATCHES,
        "track_endpoint_steps": TRACK_ENDPOINT_STEPS,
    },
}

if RUN_OPTIONAL_K3 and stage3_model is not None:
    summary_bundle["stage3_losses"] = stage3_losses
    summary_bundle["stage3_history"] = stage3_history

if RUN_OPTIONAL_K3 and stage3_distill_model is not None:
    summary_bundle["stage3_distill_losses"] = stage3_distill_losses
    summary_bundle["stage3_distill_history"] = stage3_distill_history
    summary_bundle["stage3_distill_results"] = stage3_distill_results

torch.save(summary_bundle, OUT_DIR / "notebook_summary.pt")

print("All outputs saved under:", OUT_DIR.resolve())
```

```
All outputs saved under: /content/mnist_rectified_flow_notebook_runs
```

In [ ]:

```
# Copy the current notebook run logs/results to Google Drive

from google.colab import drive
from pathlib import Path
from datetime import datetime
import shutil
import os

#Mount Google Drive
drive.mount('/content/drive')

#Set source and destination
SRC_DIR = Path("mnist_rectified_flow_notebook_runs")  # current local
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
DST_ROOT = Path("/content/drive/MyDrive/mnist_rectified_flow_notebook_
```

```
DST_DIR = DST_ROOT / f"run_{timestamp}"

#Validate source
if not SRC_DIR.exists():
    raise FileNotFoundError(f"Source folder not found: {SRC_DIR.resolv

#Create destination and copy everything
DST_ROOT.mkdir(parents=True, exist_ok=True)
shutil.copytree(SRC_DIR, DST_DIR)

#also create a latest snapshot
LATEST_DIR = DST_ROOT / "latest"
if LATEST_DIR.exists():
    shutil.rmtree(LATEST_DIR)
shutil.copytree(SRC_DIR, LATEST_DIR)

#Print summary
all_files = [p for p in DST_DIR.rglob("*") if p.is_file()]
total_bytes = sum(p.stat().st_size for p in all_files)

print(f"Copied logs to: {DST_DIR}")
print(f"Updated latest snapshot: {LATEST_DIR}")
print(f"Total files copied: {len(all_files)}")
print(f"Total size: {total_bytes / (1024**2):.2f} MB")
```

```
Mounted at /content/drive
Copied logs to: /content/drive/MyDrive/mnist_rectified_flow_notebook_ru
ns_backups/run_20260419_024430
Updated latest snapshot: /content/drive/MyDrive/mnist_rectified_flow_no
tebook_runs_backups/latest
Total files copied: 159
Total size: 424.81 MB
```